

Block 4: File Structures and Advanced Data Structures

Short explanations, worked examples and self-check questions for every unit, with every idea also explained in Malayalam.

ഓരോ ആശയവും ലളിതമായ ഇംഗ്ലീഷിലും മലയാളത്തിലും, ഉദാഹരണങ്ങളോടെ.

UNIT 10

Hashing

ഒരു key-യെ hash function ഉപയോഗിച്ച് table-ലെ ഒരു index ആക്കി മാറ്റി, ശരാശരി $O(1)$ സമയത്തിൽ data സൂക്ഷിക്കാനും തിരികെ എടുക്കാനുമുള്ള രീതിയാണ് hashing. Index mapping, hash function, collision പരിഹരിക്കുന്ന രീതികൾ (chaining, linear/quadratic probing, double hashing), load factor, rehashing എന്നിവയാണ് ഈ unit-ൽ.

Hashing and why we use it

Index mapping (trivial hashing)

Challenges with index mapping

Hash function

Simple hash example in C

Collision and separate chaining

Open addressing: linear probing

Quadratic probing

Double hashing

Comparison of collision resolution methods

Load factor

Rehashing

Hashing and why we use it

Hashing: a technique that uses a hash function to convert a key into an index (hash value) of a table, so the record can be stored and found in $O(1)$ average time.

Hash table: an array in which each record is stored at the index given by the hash value of its key.

Hashing: ഒരു hash function ഉപയോഗിച്ച് key-യെ ഒരു table-ന്റെ index (hash value) ആക്കി മാറ്റി, record ശരാശരി $O(1)$ സമയത്തിൽ സൂക്ഷിക്കാനും കണ്ടെത്താനും സഹായിക്കുന്ന technique.

Hash table: ഓരോ record-ഉം അതിന്റെ key-യുടെ hash value പറയുന്ന index-ൽ സൂക്ഷിക്കുന്ന array.

Linear search takes $O(n)$ and binary search $O(\log n)$. Hashing aims at $O(1)$: we compute where a key should be and go straight there, whatever the total number of keys. IGNOU lists three drivers for hashing: (1) constant-time retrieval and insertion, (2) efficient handling of a large set of input keys, and (3) cheap (cost-efficient) computation of the hash key.

മലയാളത്തിൽ

ഒരു apartment-ൽ flat number അറിയാമെങ്കിൽ നമ്മൾ ഓരോ വാതിലിലും മുട്ടി നോക്കേണ്ട; നേരെ ആ flat-ലേക്ക് പോകും. Hashing-ഉം ഇതുതന്നെ: key-ൽ നിന്ന് ഒരു ചെറിയ കണക്ക് (hash function) ചെയ്ത് "ഈ key ഏത് slot-ൽ" എന്ന് കണ്ടുപിടിക്കുന്നു, എന്നിട്ട് നേരെ ആ slot-ൽ പോയി നോക്കുന്നു. അതുകൊണ്ട് എത്ര data ഉണ്ടെങ്കിലും search സമയം ഏകദേശം ഒരുപോലെ.

key $\rightarrow$ hash function $h \rightarrow$ hash value $h(\text{key}) \rightarrow$ slot in the hash table

Linear search $O(n)$ | Binary search $O(\log n)$ | Hashing $O(1)$ on average

EXAMPLE

A hash table of size 10 uses $h(x) = x \bmod 10$. Where is key 4562 stored, and how do we search for it?

$h(4562) = 4562 \bmod 10 = 2$. Store it in slot 2. To search, compute $h(4562) = 2$ again and look only in slot 2: one computation and one array access, no matter how many keys are in the table.

Index mapping (trivial hashing)

Index mapping (trivial hashing / direct addressing): using the key value itself as the array index, so the record with key k is stored in $\text{array}[k]$.

Index mapping (trivial hashing / direct addressing): key value തന്നെ array index ആയി ഉപയോഗിക്കുന്ന രീതി; key k ഉള്ള record $\text{array}[k]$ -ൽ സൂക്ഷിക്കുന്നു.

Take a very large array and use the key as the index. Array indexing is $O(1)$, so storing and finding are both constant time. IGNOU's example stores user names by 4-digit employee id: the array must have a slot for every possible id up to the largest one (9875 in Figure 1). Index mapping is suitable when all keys are known or predictable and small: months of a year (12 slots), days of a month (31), states of a country.

മലയാളത്തിൽ

School-ലെ roll number 1 മുതൽ 40 വരെ ആണെങ്കിൽ 40 കസേര നിറത്തി, roll number 7 ഉള്ള കുട്ടി കസേര 7-ൽ ഇരിക്കും എന്ന് പറയുന്നതുപോലെ. Key തന്നെ index. ആരെയെങ്കിലും വിളിക്കാൻ നേരെ ആ കസേരയിൽ നോക്കിയാൽ മതി, $O(1)$. Key-കൾ ചെറുതും മുൻകൂട്ടി അറിയാവുന്നതും ആകുമ്പോൾ മാത്രമാണ് ഇത് നല്ലത്.

store: $A[k] = \text{record}$ | search: return $A[k]$ | delete: $A[k] = \text{empty}$

Time: $O(1)$ for each operation

Space: array size $\geq$ largest key + 1

EXAMPLE

Store the number of days in each month so that month number m (1 to 12) gives its days in $O(1)$.

Use an array `days[13]` and ignore index 0: `days[1] = 31`, `days[2] = 28`, ..., `days[12] = 31`. The key (month number) is the index, so `days[4]` gives 30 immediately.

Exam tip: Check Your Progress asks "___ provides direct mapping of keys to the hash table index" (answer: index mapping) and "time complexity of index mapping is ___" (answer: $O(1)$).

Challenges with index mapping

Limitations of index mapping: keys must be non-negative integers, and the array must be as large as the largest key, which wastes space when keys are few or spread out.

Index mapping-ന്റെ പരിമിതികൾ: key-കൾ non-negative integer ആയിരിക്കണം, കൂടാതെ array ഏറ്റവും വലിയ key-യുടെ അത്രയും വലുതായിരിക്കണം; key-കൾ കുറവോ ചിതറിയതോ ആണെങ്കിൽ ധാരാളം space പാഴാകും.

Two drawbacks: (1) only non-negative integers can be indexes, so strings, images or alphanumeric ids cannot be used directly; (2) the array must be sized for the largest key. With 5 employees whose ids go up to 9875, we need 9876 slots and 9871 of them stay empty. This hurts performance and scalability. The fix is a conversion function that turns any key into a non-negative integer and squeezes it into a small range. That function is the **hash function**.

മലയാളത്തിൽ

5 പേരുടെ വിവരം സൂക്ഷിക്കാൻ 9876 അലമാര വാങ്ങുന്നത് പോലെയാണിത്, ബാക്കിയെല്ലാം കാലി. കൂടാതെ "RAHUL" പോലുള്ള പേര് നേരിട്ട് array index ആക്കാൻ പറ്റില്ല. അതുകൊണ്ട് ഏത് key-യെയും ഒരു ചെറിയ non-negative number ആക്കുന്ന ഒരു function വേണം; അതാണ് hash function.

EXAMPLE

Employee ids 9875, 1234, 51, 821, 4562. Compare index mapping with $h(x) = x \bmod 10$.

Index mapping needs an array of 9876 slots (0 to 9875) for 5 records: $5/9876 \approx 0.05\%$ used. With $h(x) = x \bmod 10$ the ids go to slots 5, 4, 1, 1, 2 of an array of size 10. The array is small, but 51 and 821 both want slot 1: this is a **collision**, handled from topic 6 onwards.

Exam tip: The two challenges asked in Check Your Progress: handling non-integer keys and large hash table size (wasted space).

Hash function

Hash function: a function that maps a key of any type to a non-negative integer in the range 0 to $m - 1$, which is used as the index in a hash table of size m .

Hash function: ഏത് type key-യെയും 0 മുതൽ $m - 1$ വരെയുള്ള ഒരു non-negative integer ആക്കി മാറ്റുന്ന function; ഈ number ആണ് size m ഉള്ള hash table-ലെ index.

IGNOU's hash function is the division (modulo) method $h(x) = x \bmod m$, where m is the table size; the remainder is always between 0 and $m - 1$. For a non-integer key such as a string, first turn it into a number (for example, from the ASCII codes of its characters) and then apply the hash function. A good hash function has four characteristics:

- **Computing efficiency:** fast to compute, even for large keys.
- **Uniform distribution:** spreads keys evenly over the table.
- **Deterministic:** always gives the same hash value for the same key.
- **Minimal collisions:** different keys should rarely get the same slot.

മലയാളത്തിൽ

Post office-ൽ PIN code നോക്കി letter ഏത് അറയിൽ ഇടണം എന്ന് തീരുമാനിക്കുന്നതുപോലെ. നിയമം വേഗത്തിൽ ചെയ്യാവുന്നതാകണം (computing efficiency), എല്ലാ അറയിലും ഏകദേശം ഒരുപോലെ letter വരണം (uniform distribution), ഒരു PIN-ന് എന്നും ഒരു അറ കിട്ടണം (deterministic), രണ്ട് letter ഒരേ അറയിലേക്ക് പോകുന്നത് കുറവാകണം (minimal collisions).

Division method: $h(x) = x \bmod m, \quad 0 \leq h(x) \leq m - 1$

String key: number = sum (or weighted sum) of ASCII codes, then apply mod m

Good choice of m: a prime not close to a power of 2

EXAMPLE 1

Hash the keys 9875, 1234, 51, 821 and 4562 into a table of size 10 with $h(x) = x \bmod 10$.

Key x	9875	1234	51	821	4562
$h(x) = x \bmod 10$	5	4	1	1	2

An array of size 10 replaces an array of size 9876. Note that 51 and 821 collide at slot 1.

EXAMPLE 2

Hash the strings "CAT" and "ACT" by adding ASCII codes, then taking mod 10.

C = 67, A = 65, T = 84. "CAT": $67 + 65 + 84 = 216$, $216 \bmod 10 = 6$. "ACT": $65 + 67 + 84 = 216$, also slot 6.

A plain sum gives every anagram the same value, so it collides a lot. A better string hash weights each position, e.g. $h = (h \times 31 + \text{code})$ for each character, then mod m.

Exam tip: "Hash value should always be non-negative": **True**, because it is used as an array index. In C and Java, $-13 \% 10$ is -3 , not 7. For keys that can be negative write $((x \% m) + m) \% m$.

Simple hash example in C

Basic hash table operations: insert (add a value at slot $h(\text{key})$), search (return the value at slot $h(\text{key})$) and delete (mark slot $h(\text{key})$ empty), each $O(1)$ when there are no collisions.

Hash table-ന്റെ അടിസ്ഥാന operations: insert (slot $h(\text{key})$ -ൽ value ചേർക്കുക), search (slot $h(\text{key})$ -ലെ value തിരികെ നൽകുക), delete (slot $h(\text{key})$ empty ആക്കുക); collision ഇല്ലെങ്കിൽ ഓരോന്നും $O(1)$.

IGNOU gives this example in Java with four functions: `getHashValue`, `addValue`, `getValue` and `removeValue`. It assumes there are **no collisions**. Below is the same design in C. Every operation first computes the hash value and then does one array access.

മലയാളത്തിൽ

എല്ലാ function-ഉം ആദ്യം hash value കണക്കാക്കുന്നു, എന്നിട്ട് ആ ഒരൊറ്റ slot-ൽ ജോലി ചെയ്യുന്നു. Collision ഇല്ല എന്ന് സങ്കൽപ്പിച്ചിരിക്കുന്നു; രണ്ട് key ഒരേ slot-ൽ വന്നാൽ പഴയ value മാഞ്ഞുപോകും. അത് പരിഹരിക്കാനാണ് അടുത്ത topic-കളിലെ രീതികൾ.

```

#include <stdio.h>

#define SIZE 10
#define EMPTY -1

int table[SIZE];          /* stores the value for each slot */

int hashValue(int key) {
    return ((key % SIZE) + SIZE) % SIZE;  /* never negative */
}

void addValue(int key, int value) {
    table[hashValue(key)] = value;
}

int getValue(int key) {
    return table[hashValue(key)];
}

void removeValue(int key) {
    table[hashValue(key)] = EMPTY;
}

int main(void) {
    int i;
    for (i = 0; i < SIZE; i++) table[i] = EMPTY;
    addValue(9875, 500);
    addValue(1234, 300);
    addValue(-13, 700);
    printf("h(9875)=%d value=%d\n", hashValue(9875), getValue(9875));
    printf("h(1234)=%d value=%d\n", hashValue(1234), getValue(1234));
    printf("h(-13)=%d value=%d\n", hashValue(-13), getValue(-13));
    removeValue(1234);
    printf("after remove: value at h(1234)=%d\n", getValue(1234));
    return 0;
}

```

```

h(9875)=5 value=500
h(1234)=4 value=300
h(-13)=7 value=700
after remove: value at h(1234)=-1

```

Exam tip: IGNOU's Java code has two bugs. (1) `addValue(int inputKey V inputValue)` is missing the comma between parameters; it should be `addValue(int inputKey, V inputValue)`. (2) `inputKey % this.hashArray.length` is negative for a negative key, which crashes with an out-of-bounds index. The C version above uses `((key % SIZE) + SIZE) % SIZE`, so `-13` goes to slot 7.

Collision and separate chaining

Collision: the situation where two different keys get the same hash value, i.e. $h(x_1) = h(x_2)$ with $x_1 \neq x_2$.

Separate chaining: a collision resolution method in which each slot of the hash table holds a linked list of all keys that hash to that slot.

Collision: രണ്ട് വ്യത്യസ്ത key-കൾക്ക് ഒരേ hash value കിട്ടുന്ന അവസ്ഥ, അതായത് $x_1 \neq x_2$ ആയിട്ടും $h(x_1) = h(x_2)$.

Separate chaining: hash table-ലെ ഓരോ slot-ഉം ആ slot-ലേക്ക് hash ചെയ്യുന്ന എല്ലാ key-കളുടെയും linked list സൂക്ഷിക്കുന്ന collision resolution രീതി.

With $h(x) = x \bmod 10$, keys 24 and 4874 both give 4: they collide. The unit covers three collision resolution techniques: separate chaining, open addressing (linear and quadratic probing) and double hashing. In separate chaining each slot points to a linked list, and colliding keys are simply added to that list. Keys (list nodes) live outside the array itself.

മലയാളത്തിൽ

ഒരു bus stop-ൽ ഒരു ബസ്സിന് ഒരു bay മാത്രം; രണ്ട് ബസ് ഒരേ bay-ൽ വന്നാൽ അത് collision. Separate chaining-ൽ ഓരോ bay-യുടെ പിന്നിലും ഒരു queue ഉണ്ടാകും: ഒരേ slot-ൽ വരുന്ന key-കൾ എല്ലാം ആ slot-ലെ linked list-ൽ ഒന്നിനു പിറകെ ഒന്നായി ചേർക്കുന്നു. Search ചെയ്യുമ്പോൾ ആ ഒരൊറ്റ list മാത്രം നോക്കിയാൽ മതി.

Insert k: 1. compute $h(k)$ 2. go to slot $h(k)$ 3. add k to the linked list at that slot

Search k: 1. compute $h(k)$ 2. go to slot $h(k)$ 3. compare k with every node of that list

Delete k: search k in the list at slot $h(k)$ and unlink its node

Average search time $O(1 + \alpha)$, $\alpha = n/m$ (load factor); worst case $O(n)$ when all keys fall in one list

EXAMPLE 1 (IGNOU, FIGURE 3)

With $h(x) = x \bmod 10$, user 2 has id 0051 and user 5 has id 821. Where are they stored?

$51 \bmod 10 = 1$ and $821 \bmod 10 = 1$. Both go to slot 1: slot 1 $\rightarrow$ user2 (51) $\rightarrow$ user5 (821) $\rightarrow$ NULL.

EXAMPLE 2

Insert 50, 700, 76, 85, 92, 73, 101 into a chained hash table of size 7 with $h(x) = x \bmod 7$. Each new key is added at the front of its list.

h values: 50 $\rightarrow$ 1, 700 $\rightarrow$ 0, 76 $\rightarrow$ 6, 85 $\rightarrow$ 1, 92 $\rightarrow$ 1, 73 $\rightarrow$ 3, 101 $\rightarrow$ 3.

Slot	Chain (front first)
0	700
1	92 $\rightarrow$ 85 $\rightarrow$ 50
2	empty
3	101 $\rightarrow$ 73
4	empty
5	empty
6	76

Search 64: $h(64) = 64 \bmod 7 = 1$. Compare with 92, 85, 50: no match, so 64 is not present (3 comparisons).

If you add each key at the end of the list instead, slot 1 reads 50 $\rightarrow$ 85 $\rightarrow$ 92; both are accepted.

```

#include <stdio.h>
#include <stdlib.h>

#define M 7

struct node {
    int key;
    struct node *next;
};

struct node *table[M];          /* each slot is the head of a linked list */

int h(int key) { return key % M; }

void insert(int key) {
    struct node *p = malloc(sizeof *p);
    if (p == NULL) { printf("No memory\n"); exit(1); }
    p->key = key;
    p->next = table[h(key)];    /* add at the front of the list */
    table[h(key)] = p;
}

int search(int key) {
    struct node *p;
    for (p = table[h(key)]; p != NULL; p = p->next)
        if (p->key == key) return 1;
    return 0;
}

void display(void) {
    int i;
    struct node *p;
    for (i = 0; i < M; i++) {
        printf("%d:", i);
        for (p = table[i]; p != NULL; p = p->next) printf(" %d ->", p->key);
        printf(" NULL\n");
    }
}

int main(void) {
    int keys[] = {50, 700, 76, 85, 92, 73, 101};
    int i;
    for (i = 0; i < 7; i++) insert(keys[i]);
    display();
    printf("search 92: %s\n", search(92) ? "found" : "not found");
    printf("search 64: %s\n", search(64) ? "found" : "not found");
    return 0;
}

```

```
0: 700 -> NULL
1: 92 -> 85 -> 50 -> NULL
2: NULL
3: 101 -> 73 -> NULL
4: NULL
5: NULL
6: 76 -> NULL
search 92: found
search 64: not found
```

Exam tip: The unit's text says 0051 and 821 hash to 1 and then says they are chained "for the slot 0". That is a slip: $51 \bmod 10 = 821 \bmod 10 = 1$, so they are chained at **slot 1**. For "___ data structure can be used to chain multiple values", answer **linked list**.

Open addressing: linear probing

Open addressing: a collision resolution method that stores every key inside the hash table itself and, on a collision, probes other slots until an empty one is found.

Linear probing: open addressing in which the i -th probe is $h(x, i) = (h(x) + i) \bmod m$, i.e. the next slots are tried one after another.

Primary clustering: the formation of long runs of consecutive occupied slots in linear probing, which makes later insertions and searches slower.

Open addressing: എല്ലാ key-കളും hash table-നുള്ളിൽ തന്നെ സൂക്ഷിക്കുകയും, collision വന്നാൽ ഒരു empty slot കിട്ടുന്നതുവരെ മറ്റ് slot-കൾ probe ചെയ്യുകയും ചെയ്യുന്ന collision resolution രീതി.

Linear probing: i -ാമത്തെ probe $h(x, i) = (h(x) + i) \bmod m$ ആയ open addressing; അടുത്തടുത്ത slot-കൾ ഒന്നൊന്നായി നോക്കുന്നു.

Primary clustering: linear probing-ൽ തുടർച്ചയായ occupied slot-കളുടെ നീണ്ട നിര (cluster) രൂപപ്പെടുകയും, അത് പിന്നീടുള്ള insert-ഉം search-ഉം മന്ദഗതിയിലാക്കുകയും ചെയ്യുന്ന പ്രശ്നം.

In open addressing there are no lists: if the home slot $h(x)$ is taken, we look for the next open (unused) slot using a probe sequence $i = 0, 1, 2, \dots$. Linear probing steps one slot at a time and wraps around with mod m . The operations change as follows:

- **Search (getHashValue):** start at $h(x)$ and check the following slots until the key is found or an empty slot is reached.
- **Insert (putHashValue):** start at $h(x)$ and move on until an empty slot is found or the table is full.

- **Delete (deleteHashValue):** put null in the slot, or better, an availability marker (occupied / available, often called DELETED).

മലയാളത്തിൽ

Cinema theatre-ൽ നിങ്ങളുടെ seat-ൽ ആരെങ്കിലും ഇരിക്കുന്നുണ്ടെങ്കിൽ തൊട്ടടുത്ത കാലി seat-ൽ ഇരിക്കുന്നതുപോലെയാണ് linear probing. പ്രശ്നം: കുറേ പേർ ഇങ്ങനെ ചെയ്യുമ്പോൾ ഒരു നിരയിൽ തുടർച്ചയായി seat നിറയും (cluster). പിന്നെ വരുന്നയാൾ ആ മുഴുവൻ നിരയും കടന്നുപോകണം. ഇതാണ് primary clustering.

$$h(x, i) = (h(x) + i) \bmod m, \quad i = 0, 1, 2, \dots, m - 1$$

Probe sequence: $h(x), h(x)+1, h(x)+2, \dots$ (wrapping round after $m - 1$)

Load factor $\alpha \leq 1$ (all keys are inside the table)

EXAMPLE 1 (IGNOU, FIGURE 4)

Insert 56, 1072, 97, 84, 60 into a table of size 5 with $h(x) = x \bmod 5$ and linear probing.

$56 \bmod 5 = 1 \rightarrow$ slot 1. $1072 \bmod 5 = 2 \rightarrow$ slot 2. $97 \bmod 5 = 2$, taken; try 3 $\rightarrow$ slot 3. $84 \bmod 5 = 4 \rightarrow$ slot 4. $60 \bmod 5 = 0 \rightarrow$ slot 0.

Slot	0	1	2	3	4
Key	60	56	1072	97	84

EXAMPLE 2

Insert 89, 18, 49, 58, 69 into a table of size 10 with $h(x) = x \bmod 10$ and linear probing.

Key	$h(x)$	Slots probed	Placed at
89	9	9	9
18	8	8	8
49	9	9 (89), 0	0
58	8	8 (18), 9 (89), 0 (49), 1	1
69	9	9 (89), 0 (49), 1 (58), 2	2

Slot	0	1	2	3	4	5	6	7	8	9
Key	49	58	69	–	–	–	–	–	18	89

Slots 8, 9, 0, 1, 2 form one cluster of 5. Any new key hashing to 8, 9, 0, 1 or 2 must walk to slot 3, and the cluster keeps growing: primary clustering.

EXAMPLE 3 (WHY DELETION NEEDS A MARKER)

In the table of Example 2, delete 49, then search 69.

If 49's slot 0 is simply made empty: search 69 starts at 9 (89, no), goes to 0, finds empty and stops with "not found", although 69 is in slot 2. Wrong.

If slot 0 is marked DELETED: the search skips it, checks 1 (58), then 2 (69): found. A DELETED slot can be reused by a later insert.

The program below implements insert, search and delete (with a DELETED marker) for all three open-addressing methods of this unit. method 1 = linear, 2 = quadratic, 3 = double hashing with $h_2(x) = 7 - (x \bmod 7)$.

```

#include <stdio.h>

#define M 10
#define EMPTY -1
#define DELETED -2

int table[M];

int h1(int x) { return x % M; }
int h2(int x) { return 7 - (x % 7); } /* never 0 */

/* i-th probe; method: 1 = linear, 2 = quadratic, 3 = double hashing */
int probe(int x, int i, int method) {
    if (method == 1) return (h1(x) + i) % M;
    if (method == 2) return (h1(x) + i * i) % M;
    return (h1(x) + i * h2(x)) % M;
}

int insert(int x, int method) {
    int i, s;
    for (i = 0; i < M; i++) {
        s = probe(x, i, method);
        if (table[s] == EMPTY || table[s] == DELETED) {
            table[s] = x;
            return s;
        }
    }
    return -1; /* no free slot found */
}

int search(int x, int method) {
    int i, s;
    for (i = 0; i < M; i++) {
        s = probe(x, i, method);
        if (table[s] == EMPTY) return -1; /* a never-used slot ends the search */
        if (table[s] == x) return s; /* DELETED slots are skipped */
    }
    return -1;
}

void removeKey(int x, int method) {
    int s = search(x, method);
    if (s != -1) table[s] = DELETED;
}

void show(void) {
    int i;
    for (i = 0; i < M; i++) {
        if (table[i] == EMPTY) printf(" [%d] -", i);
        else if (table[i] == DELETED) printf(" [%d] D", i);
        else printf(" [%d] %d", i, table[i]);
    }
}

```

```

    }
    printf("\n");
}

int main(void) {
    int keys[] = {89, 18, 49, 58, 69};
    char *name[] = {"", "Linear", "Quadratic", "Double"};
    int m, i;
    for (m = 1; m <= 3; m++) {
        for (i = 0; i < M; i++) table[i] = EMPTY;
        for (i = 0; i < 5; i++) insert(keys[i], m);
        printf("%s:", name[m]);
        show();
    }
    /* delete with a marker (linear probing) */
    for (i = 0; i < M; i++) table[i] = EMPTY;
    for (i = 0; i < 5; i++) insert(keys[i], 1);
    removeKey(49, 1);
    show();
    printf("search 69 -> slot %d\n", search(69, 1));
    return 0;
}

```

```

Linear: [0] 49 [1] 58 [2] 69 [3] - [4] - [5] - [6] - [7] - [8] 18 [9] 89
Quadratic: [0] 49 [1] - [2] 58 [3] 69 [4] - [5] - [6] - [7] - [8] 18 [9] 89
Double: [0] 69 [1] - [2] - [3] 58 [4] - [5] - [6] 49 [7] - [8] 18 [9] 89
        [0] D [1] 58 [2] 69 [3] - [4] - [5] - [6] - [7] - [8] 18 [9] 89
search 69 -> slot 2

```

Exam tip: In a probing trace, always show the slots you checked, not just the final table. For delete, say that open addressing needs a **DELETED marker** (the unit's "availability marker"), because setting the slot to null breaks later searches.

Quadratic probing

Quadratic probing: open addressing in which the i -th probe is $h(x, i) = (h(x) + i^2) \bmod m$, so the jumps grow as 1, 4, 9, ... from the home slot.

Secondary clustering: keys with the same home slot follow exactly the same probe sequence, so they keep colliding with each other.

Quadratic probing: i -ാമത്തെ probe $h(x, i) = (h(x) + i^2) \bmod m$ ആയ open addressing; home slot-ൽ നിന്ന് 1, 4, 9, ... എന്നിങ്ങനെ ചാട്ടം വലുതാകുന്നു.

Secondary clustering: ഒരേ home slot ഉള്ള key-കൾ കൃത്യം ഒരേ probe sequence പിന്തുടരുന്നതുകൊണ്ട് അവ വീണ്ടും വീണ്ടും പരസ്പരം collide ചെയ്യുന്ന പ്രശ്നം.

Quadratic probing makes larger and larger jumps ($h, h+1, h+4, h+9, \dots$), so keys do not pile up in one continuous block and primary clustering is avoided. But the probe sequence depends only on $h(x)$: every key with the same home slot tries the same slots in the same order. That is secondary clustering, a milder problem.

മലയാളത്തിൽ

Linear probing-ൽ തൊട്ടടുത്ത seat നോക്കുന്നു; quadratic probing-ൽ 1, 4, 9 seat അകലേക്ക് ചാടുന്നു. അതുകൊണ്ട് ഒരു നിര മുഴുവൻ നിറയുന്ന പ്രശ്നം (primary clustering) ഇല്ല. പക്ഷേ ഒരേ seat number ഉള്ള എല്ലാവരും ഒരേ വഴിക്കാണ് ചാടുന്നത്; അവർ തമ്മിൽ വീണ്ടും മുട്ടും. ഇതാണ് secondary clustering.

$$h(x, i) = (h(x) + i^2) \bmod m, \quad i = 0, 1, 2, \dots$$

Probe sequence: $h, h+1, h+4, h+9, h+16, \dots$ (all mod m)

If m is prime and $\alpha \leq 0.5$, a free slot is always found

EXAMPLE 1

Insert 89, 18, 49, 58, 69 into a table of size 10 with $h(x) = x \bmod 10$ and quadratic probing.

Key	$h(x)$	Probes (i : slot)	Placed at
89	9	0: 9	9
18	8	0: 8	8
49	9	0: 9 (89), 1: $(9+1) \bmod 10 = 0$	0
58	8	0: 8 (18), 1: 9 (89), 2: $(8+4) \bmod 10 = 2$	2
69	9	0: 9 (89), 1: 0 (49), 2: $(9+4) \bmod 10 = 3$	3

Slot	0	1	2	3	4	5	6	7	8	9
Key	49	–	58	69	–	–	–	–	18	89

Compare with linear probing: 58 and 69 now jump out of the cluster, leaving slot 1 free.

EXAMPLE 2 (QUADRATIC PROBING CAN MISS FREE SLOTS)

Table size 10, a key with $h(x) = 0$. Which slots can quadratic probing ever reach?

$i^2 \bmod 10$ for $i = 0, 1, 2, \dots$ gives 0, 1, 4, 9, 6, 5, 6, 9, 4, 1, 0, ... so only slots {0, 1, 4, 5, 6, 9} are ever tried. Slots 2, 3, 7, 8 are never probed, so an insert can fail even though the table has free space. That is why m is chosen prime and the table is kept at most half full.

Exam tip: The extracted text prints the formula as " $h(x)+i^2$ "; it means i squared: $h(x, i) = (h(x) + i^2) \bmod m$. One-line answers from Check Your Progress: linear probing → **primary** clustering; **quadratic** probing makes large jumps and leads to secondary clustering.

Double hashing

Double hashing: open addressing that uses a second hash function h_2 as the step size: $h(x, i) = (h_1(x) + i \cdot h_2(x)) \bmod m$, where $h_2(x)$ is never 0.

Double hashing: രണ്ടാമതൊരു hash function h_2 step size ആയി ഉപയോഗിക്കുന്ന open addressing: $h(x, i) = (h_1(x) + i \cdot h_2(x)) \bmod m$; ഇവിടെ $h_2(x)$ ഒരിക്കലും 0 ആകരുത്.

h_1 gives the starting slot; h_2 , a different function of the key, gives the jump size. Two keys with the same home slot usually have different h_2 values, so they follow different probe sequences. This removes both primary and

secondary clustering. h_2 must never be 0 (otherwise the probe never moves), and it should be relatively prime to m (easy when m is prime) so that every slot can be reached.

മലയാളത്തിൽ

ഒരേ stop-ൽ ഇറങ്ങിയ രണ്ടുപേർ വീട് തേടി നടക്കുന്നു എന്ന് കരുതുക: ഒരാൾ 3 വീട് വീതം, മറ്റൊരാൾ 5 വീട് വീതം ചാടുന്നു. അവരുടെ വഴി വ്യത്യസ്തമായതിനാൽ വീണ്ടും മുട്ടാനുള്ള സാധ്യത കുറവ്. ഇതാണ് double hashing: ചാട്ടത്തിന്റെ വലിപ്പം key-യെ ആശ്രയിച്ച് h_2 തീരുമാനിക്കുന്നു. $h_2 = 0$ ആയാൽ ആൾ ഒരടി പോലും നീങ്ങില്ല, അതുകൊണ്ട് അത് ഒഴിവാക്കണം.

$$h(x, i) = (h_1(x) + i \cdot h_2(x)) \bmod m, \quad i = 0, 1, 2, \dots$$

Safe choices of h_2 : $h_2(x) = R - (x \bmod R)$ with prime $R < m$, or $h_2(x) = 1 + (x \bmod (m - 1))$

Requirement: $h_2(x) \neq 0$ and $\gcd(h_2(x), m) = 1$

EXAMPLE 1 (IGNOU, FIGURE 5)

Table size 5, $h_1(x) = x \bmod 5$, $h_2(x) = x \bmod 7$. Insert 60 and 80.

60: $h_1 = 0$, slot 0 is free $\rightarrow$ slot 0.

80: $h_1 = 0$, collision. $h_2(80) = 80 \bmod 7 = 3$. $i = 1$: $(0 + 1 \times 3) \bmod 5 = 3$, free $\rightarrow$ slot 3.

Slot	0	1	2	3	4
Key	60	-	-	80	-

Now try to insert 70: $h_1(70) = 0$ (taken) and $h_2(70) = 70 \bmod 7 = 0$, so every probe is $(0 + i \times 0) \bmod 5 = 0$. The insert loops for ever on slot 0.

EXAMPLE 2

Insert 89, 18, 49, 58, 69 into a table of size 10 with $h_1(x) = x \bmod 10$ and $h_2(x) = 7 - (x \bmod 7)$.

Key	h_1	h_2	Probes	Placed at
89	9	–	9	9
18	8	–	8	8
49	9	$7 - 0 = 7$	$9 (89), (9 + 7) \bmod 10 = 6$	6
58	8	$7 - 2 = 5$	$8 (18), (8 + 5) \bmod 10 = 3$	3
69	9	$7 - 6 = 1$	$9 (89), (9 + 1) \bmod 10 = 0$	0

Slot	0	1	2	3	4	5	6	7	8	9
Key	69	–	–	58	–	–	49	–	18	89

49 and 69 share home slot 9 but use different steps (7 and 1), so they end up far apart. The "Double" line of the program output in topic 7 shows the same table.

Exam tip: IGNOU says h_2 must "yield non-zero value" (and the answer to "second hash function in double hashing can result in 0" is **False**), yet its own example uses $h_2(x) = x \bmod 7$, which is 0 for 70, 140, 7, ... and then the probe never moves. For $m = 5$ use, for example, $h_2(x) = 3 - (x \bmod 3)$ (range 1 to 3): it is never 0, and every step 1 to 3 is relatively prime to 5. (A step of 5, e.g. from $7 - (x \bmod 7)$, would also get stuck on one slot when $m = 5$.)

Comparison of collision resolution methods

Collision resolution technique: a method for placing a key whose home slot is already occupied; the main ones are separate chaining and open addressing (linear probing, quadratic probing, double hashing).

Collision resolution technique: home slot നേരത്തെ occupied ആയ ഒരു key-ക്ക് സ്ഥലം കണ്ടെത്താനുള്ള രീതി; പ്രധാനപ്പെട്ടവ separate chaining-ഉം open addressing-ഉം (linear probing, quadratic probing, double hashing) ആണ്.

The first three rows are IGNOU's table; the last two rows follow from the earlier topics and are often asked in long answers.

Feature	Separate chaining	Linear probing	Quadratic probing	Double hashing
Primary clustering	No	Yes	No	No
Secondary clustering	No	No	Yes	No
Key storage	Inside and outside the table (linked lists)	Inside the table	Inside the table	Inside the table
Load factor α	Can exceed 1	≤ 1	≤ 1 (keep ≤ 0.5)	≤ 1
Deletion	Easy: unlink the node	Needs a DELETED marker		

മലയാളത്തിൽ

ചുരുക്കത്തിൽ: chaining-ൽ clustering ഇല്ല, table നിറഞ്ഞാലും list നീട്ടാം, പക്ഷേ pointer-നായി അധിക memory വേണം. Linear probing ലളിതമാണ്, പക്ഷേ primary clustering ഉണ്ട്. Quadratic probing അത് ഒഴിവാക്കുന്നു, പക്ഷേ secondary clustering വരും. Double hashing രണ്ടും ഒഴിവാക്കുന്നു, പകരം രണ്ടാമതൊരു hash കണക്കാക്കണം.

EXAMPLE

A table must hold keys whose number is unknown and may grow beyond the table size. Which method fits, and why?

Separate chaining. In open addressing all keys are inside the table, so it can never hold more than m keys ($\alpha \leq 1$). In chaining the lists grow outside the array, so α can exceed 1; performance just degrades gradually as the lists get longer.

Exam tip: The text says open addressing has "mainly two variants", linear and quadratic probing, but its answer key lists three main techniques: **linear probing, quadratic probing and double hashing**. Double hashing is also open addressing, so write all three.

Load factor

Load factor (α): the ratio of the number of keys stored in the hash table to the table size, $\alpha = n/m$.

Load factor (α): hash table-ൽ സൂക്ഷിച്ചിരിക്കുന്ന key-കളുടെ എണ്ണവും table size-ഉം തമ്മിലുള്ള അനുപാതം, $\alpha = n/m$.

As the table fills up, collisions increase and operations stop being constant time. A threshold on the load factor (commonly 0.75) tells us when to enlarge the table: once α goes above the threshold, the table is resized, normally to double its size. In open addressing $\alpha \leq 1$ because every key is inside the table; in separate chaining α can be greater than 1.

മലയാളത്തിൽ

ഒരു KSRTC ബസ്സിൽ 50 seat ഉണ്ട്, 40 പേർ കയറി: load factor $40/50 = 0.8$. ബസ് നിറയുമ്പോൾ seat കണ്ടെത്താൻ കൂടുതൽ സമയം എടുക്കും. അതുകൊണ്ട് ഒരു പരിധി (ഉദാ: 0.75) കടന്നാൽ വലിയ ബസ് (ഇരട്ടി size ഉള്ള table) എടുക്കുന്നു.

Load factor $\alpha = (\text{number of keys stored } n) / (\text{table size } m)$

Open addressing: $0 \leq \alpha \leq 1$ | Separate chaining: α can be > 1

Resize (usually to $2m$) when $\alpha > \text{threshold}$

EXAMPLE 1 (IGNOU)

Table size 10, load-factor threshold 0.75. How many keys can be inserted before a resize?

7 keys: $\alpha = 7/10 = 0.70 \leq 0.75$, no resize. The 8th key makes $\alpha = 8/10 = 0.80 > 0.75$, so the table is resized to 20.

EXAMPLE 2

A chained hash table of size 5 holds 12 keys. Find α and the average chain length.

$\alpha = 12/5 = 2.4$. On average each chain has 2.4 keys, so an unsuccessful search looks at about 2.4 nodes. $\alpha > 1$ is possible only because chaining stores keys outside the array.

Exam tip: "Load factor triggers ____": **resizing of the hash table** (followed by rehashing).

Rehashing

Rehashing: after the hash table is resized (usually doubled) because the load factor crossed its threshold, recomputing the hash value of every existing key with the new size and reinserting it.

Rehashing: load factor പരിധി കടന്നതിനാൽ hash table resize ചെയ്യുശേഷം (സാധാരണ ഇരട്ടിയാക്കി), നിലവിലുള്ള ഓരോ key-യുടെയും hash value പുതിയ size ഉപയോഗിച്ച് വീണ്ടും കണക്കാക്കി പുതിയ table-ൽ ചേർക്കൽ.

Since $h(x) = x \bmod m$ depends on m , a key's slot changes when m changes, so the old positions are no longer valid. Every existing key must be hashed again into the new table. Rehashing costs $O(n)$, which is expensive for a large table; choosing a sensible initial size avoids frequent rehashing. Steps: (1) on every insert compute the load factor; (2) if it exceeds the threshold, increase the table size (normally double it); (3) recompute the hash value of each existing key and place it in the new table.

മലയാളത്തിൽ

വീട് മാറുമ്പോൾ പഴയ അലമാരയിലെ സാധനങ്ങൾ പുതിയ വലിയ അലമാരയിലേക്ക് അതേ അറ number-ൽ വെക്കാൻ പറ്റില്ല; ഓരോന്നും വീണ്ടും തരംതിരിച്ച് വെക്കണം. അതുപോലെ table size മാറിയാൽ $x \bmod m$ -ന്റെ ഉത്തരവും മാറും; അതുകൊണ്ട് എല്ലാ key-കളും വീണ്ടും hash ചെയ്യണം. ഇതാണ് rehashing. ഇത് ചെലവുള്ള ജോലിയാണ്, അതിനാൽ തുടക്കത്തിൽ തന്നെ നല്ല size തിരഞ്ഞെടുക്കണം.

After insert: if $n/m > \text{threshold}$ then $m_{\text{new}} = 2m$ and each key x moves to $h(x) = x \bmod m_{\text{new}}$
Cost of one rehash: $O(n)$

EXAMPLE (IGNOU)

Table size 4, load-factor threshold 0.60, $h(x) = x \bmod m$. Insert 30, 31, 32.

Insert	Slot (mod 4)	Load factor	Action
30	2	$1/4 = 0.25$	none
31	3	$2/4 = 0.50$	none
32	0	$3/4 = 0.75 > 0.60$	resize to 8 and rehash

Rehash with $m = 8$: $30 \bmod 8 = 6$, $31 \bmod 8 = 7$, $32 \bmod 8 = 0$. New load factor $3/8 = 0.375$.

Slot	0	1	2	3	4	5	6	7
Key	32	-	-	-	-	-	30	31

The program below reproduces this example (linear probing, threshold 0.60, doubling).

```

#include <stdio.h>
#include <stdlib.h>

#define EMPTY -1

int *table;
int size = 4, count = 0;
double maxLoad = 0.60;

void place(int *t, int m, int x) {          /* linear probing */
    int s = x % m;
    while (t[s] != EMPTY) s = (s + 1) % m;
    t[s] = x;
}

void rehash(void) {
    int i, newSize = 2 * size;
    int *t = malloc(newSize * sizeof(int));
    if (t == NULL) { printf("No memory\n"); exit(1); }
    for (i = 0; i < newSize; i++) t[i] = EMPTY;
    for (i = 0; i < size; i++)
        if (table[i] != EMPTY) place(t, newSize, table[i]);
    free(table);
    table = t;
    size = newSize;
}

void insert(int x) {
    place(table, size, x);
    count++;
    printf("insert %d: load factor = %d/%d = %.2f\n", x, count, size, (double)count / size);
    if ((double)count / size > maxLoad) {
        rehash();
        printf("  resized to %d, load factor now %.3f\n", size, (double)count / size);
    }
}

void show(void) {
    int i;
    for (i = 0; i < size; i++)
        if (table[i] != EMPTY) printf(" [%d] %d", i, table[i]);
    printf("\n");
}

int main(void) {
    int i;
    table = malloc(size * sizeof(int));
    if (table == NULL) return 1;
    for (i = 0; i < size; i++) table[i] = EMPTY;
    insert(30); show();
    insert(31); show();
}

```

```

insert(32); show();
free(table);
return 0;
}

```

```

insert 30: load factor = 1/4 = 0.25
[2] 30
insert 31: load factor = 2/4 = 0.50
[2] 30 [3] 31
insert 32: load factor = 3/4 = 0.75
    resized to 8, load factor now 0.375
[0] 32 [6] 30 [7] 31

```

Exam tip: "___ should be done for the existing keys of the hash table post resizing": **rehashing**. Mention that simply copying keys to the same index is wrong, because $x \bmod 8$ differs from $x \bmod 4$ (30 moves from slot 2 to slot 6).

Quick recap

- Hashing maps a key to a table index with a hash function, giving $O(1)$ average insert, search and delete.
- Drivers: constant-time retrieval and insertion, handling large key sets, cheap hash computation.
- Index mapping uses the key itself as the index: $O(1)$ but needs non-negative integer keys and an array as big as the largest key.
- A hash function converts any key to an index $0 \dots m - 1$; IGNOU uses $h(x) = x \bmod m$.
- Good hash function: computing efficiency, uniform distribution, deterministic, minimal collisions.
- Hash values must be non-negative; for negative keys use $((x \% m) + m) \% m$.
- Collision: two different keys get the same hash value.
- Separate chaining: each slot holds a linked list; α can exceed 1; deletion is easy.
- Linear probing: $h(x, i) = (h(x) + i) \bmod m$; causes primary clustering.
- Quadratic probing: $h(x, i) = (h(x) + i^2) \bmod m$; avoids primary but has secondary clustering; use prime m and $\alpha \leq 0.5$.
- Double hashing: $h(x, i) = (h_1(x) + i \cdot h_2(x)) \bmod m$; h_2 must never be 0; avoids both clusterings.
- Open addressing deletion needs a DELETED marker, otherwise later searches stop too early.
- Load factor $\alpha = n/m$; crossing the threshold (e.g. 0.75) triggers a resize, usually doubling.
- Rehashing recomputes every existing key's slot with the new size; it costs $O(n)$.

Check yourself

Q. 1. Fill in the blanks: (a) ____ provides direct mapping of keys to the hash table index. (b) The time complexity of index mapping is _____. (c) ____ data structure can be used to chain multiple values to the same slot.

(a) Index mapping. (b) $O(1)$. (c) Linked list.

Q. 2. What are the two main challenges with index mapping, and how does a hash function solve them?

(1) Only non-negative integer keys can be used; (2) the array must be as large as the largest key, wasting space. A hash function first converts any key (string, alphanumeric id) to a non-negative integer and then reduces it to the range $0 \dots m - 1$, so a small array of size m is enough.

Q. 3. List the characteristics of a good hash function. Is "hash value should always be non-negative" true or false?

Computing efficiency, uniform distribution, deterministic, minimal collisions. The statement is **true**: the hash value is an array index.

Q. 4. Insert 23, 43, 13, 27, 33 into a separately chained hash table of size 10 with $h(x) = x \bmod 10$ (add at the end of each list). Give the table and the load factor.

h values: $23 \rightarrow 3, 43 \rightarrow 3, 13 \rightarrow 3, 27 \rightarrow 7, 33 \rightarrow 3$.

Slot 3: $23 \rightarrow 43 \rightarrow 13 \rightarrow 33$; slot 7: 27; all other slots empty. Load factor $\alpha = 5/10 = 0.5$. Searching 33 needs 4 comparisons in slot 3's list.

Q. 5. Insert 23, 43, 13, 27, 33 into a table of size 10 with $h(x) = x \bmod 10$ using linear probing.

$23 \rightarrow 3$. $43 \rightarrow 3$ taken, 4. $13 \rightarrow 3, 4$ taken, 5. $27 \rightarrow 7$. $33 \rightarrow 3, 4, 5$ taken, 6.

Slot	0	1	2	3	4	5	6	7	8	9
Key	–	–	–	23	43	13	33	27	–	–

Slots 3 to 7 form a cluster of 5: primary clustering.

Q. 6. Repeat Question 5 with quadratic probing, $h(x, i) = (h(x) + i^2) \bmod 10$.

23 → 3. 43: 3 taken, $3 + 1 = 4$. 13: 3, 4 taken, $3 + 4 = 7$. 27: 7 taken, $7 + 1 = 8$. 33: 3, 4, 7 taken, $(3 + 9) \bmod 10 = 2$.

Slot	0	1	2	3	4	5	6	7	8	9
Key	–	–	33	23	43	–	–	13	27	–

43, 13 and 33 share home slot 3 and follow the same sequence 3, 4, 7, 2, ...: secondary clustering.

Q. 7. Repeat Question 5 with double hashing, $h_1(x) = x \bmod 10$ and $h_2(x) = 7 - (x \bmod 7)$.

23 → 3. 43: 3 taken, $h_2 = 7 - 1 = 6$, $(3 + 6) \bmod 10 = 9$. 13: 3 taken, $h_2 = 7 - 6 = 1$, slot 4. 27 → 7. 33: 3 taken, $h_2 = 7 - 5 = 2$, slot 5.

Slot	0	1	2	3	4	5	6	7	8	9
Key	–	–	–	23	13	33	–	27	–	43

Each key needed at most 2 probes, because keys with the same home slot take different steps.

Q. 8. "The second hash function in double hashing can result in 0." True or false? Explain with $h_2(x) = x \bmod 7$.

False: it must never be 0. With $h_2(x) = x \bmod 7$, key 70 gives $h_2 = 0$, so every probe $(h_1 + i \times 0) \bmod m$ is the same slot and the insert never ends. Use $h_2(x) = 7 - (x \bmod 7)$, which is always 1 to 7.

Q. 9. A hash table of size 8 uses $h(x) = x \bmod m$, linear probing and load-factor threshold 0.75 (resize by doubling). Insert 5, 13, 20, 26, 37, 42, 50. When is the table resized, and what does it look like after rehashing (reinsert in insertion order)?

Size 8: $5 \rightarrow 5$; $13 \rightarrow 5$ taken, 6; $20 \rightarrow 4$; $26 \rightarrow 2$; $37 \rightarrow 5, 6$ taken, 7; $42 \rightarrow 2$ taken, 3. After 6 keys $\alpha = 6/8 = 0.75$, not above 0.75, so no resize yet.

$50 \rightarrow 2, 3, 4, 5, 6, 7$ all taken, wraps to 0. Now $\alpha = 7/8 = 0.875 > 0.75$, so the table is doubled to 16 and rehashed.

Size 16: $5 \rightarrow 5$; $13 \rightarrow 13$; $20 \rightarrow 4$; $26 \rightarrow 10$; $37 \rightarrow 5$ taken, 6; $42 \rightarrow 10$ taken, 11; $50 \rightarrow 2$.

Slot	2	4	5	6	10	11	13
Key	50	20	5	37	26	42	13

All other slots are empty. New $\alpha = 7/16 \approx 0.44$.

Q. 10. Fill in the blanks: (a) Load factor triggers _____. (b) Linear probing leads to _____ clustering. (c) _____ should be done for the existing keys after resizing.

(a) Resizing of the hash table. (b) Primary. (c) Rehashing.

Advanced Data Structures

Binary search tree-യിൽ നിന്ന് തുടങ്ങി, സ്വയം balance ചെയ്യുന്ന scapegoat tree, string search-ന് ഉപയോഗിക്കുന്ന trie, integer keys-ന് ഉള്ള binary trie, X-fast trie, Y-fast trie എന്നിവ ഈ unit-ൽ പഠിക്കുന്നു. ഓരോന്നിലും search, insert, delete, successor, predecessor എങ്ങനെ ചെയ്യുന്നു എന്നും അവയുടെ time complexity-യും ഇവിടെ കാണാം.

Binary search tree (BST): a quick review

Scapegoat trees and α -weight balance

Operations on scapegoat trees

Tries

Binary tries

X-fast tries

Operations on X-fast tries

Y-fast tries

Operations on Y-fast tries

Binary search tree (BST): a quick review

Binary search tree: a binary tree in which, for every node, all keys in its left subtree are smaller than the node's key and all keys in its right subtree are larger.

Successor / predecessor of k : the smallest key greater than k / the largest key smaller than k .

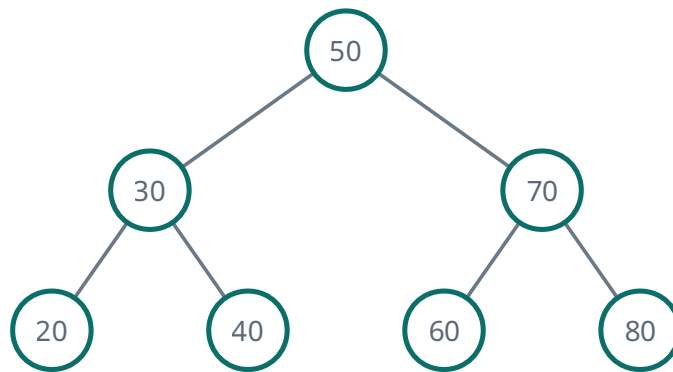
Binary search tree: ഓരോ node-ന്റെയും left subtree-യിലെ എല്ലാ keys-ഉം ആ node-ന്റെ key-യെക്കാൾ ചെറുതും right subtree-യിലെ എല്ലാ keys-ഉം വലുതും ആയ binary tree.

k -യുടെ successor / predecessor: k -യെക്കാൾ വലിയ keys-ൽ ഏറ്റവും ചെറുത് / k -യെക്കാൾ ചെറിയ keys-ൽ ഏറ്റവും വലുത്.

A BST keeps its keys in sorted order, so it is useful for both searching and sorting (an inorder traversal lists the keys in ascending order). To search for a key we start at the root: if the key is smaller we go left, if larger we go right, and we stop when we find it or fall off the tree. Every operation walks one root-to-leaf path, so its cost is $O(h)$, where h is the height. A well-shaped tree has $h \approx \log_2 n$, but if keys arrive in sorted order the tree becomes a chain with $h = n - 1$, so the **worst case** for search, insert and delete is $O(n)$. All the structures in this unit are ways to avoid that worst case or to beat $O(\log n)$ for integer keys.

മലയാളത്തിൽ

Dictionary-യിൽ ഒരു വാക്ക് തിരയുന്നതുപോലെയാണ് BST: നടുവിൽ തുറന്നു നോക്കി, വാക്ക് മുമ്പാണോ പിമ്പാണോ എന്ന് നോക്കി ഒരു വശത്തേക്ക് മാത്രം പോകുന്നു. പക്ഷേ keys ക്രമത്തിൽ (10, 20, 30, ...) insert ചെയ്താൽ tree ഒരു നീണ്ട ചങ്ങല പോലെ ആകും; അപ്പോൾ search-ന് $O(n)$ സമയം എടുക്കും. ഈ പ്രശ്നം ഒഴിവാക്കാനാണ് scapegoat tree പോലുള്ള self-balancing trees.



BST built by inserting 50, 30, 70, 20, 40, 60, 80.

Search / insert / delete in a BST: $O(h)$

Well balanced: $h \approx \log_2 n \Rightarrow O(\log n)$

Worst case (skewed chain): $h = n - 1 \Rightarrow O(n)$

EXAMPLE

In the BST above, (a) trace the search for 60 and for 65, (b) find the successor of 40 and the predecessor of 60.

(a) 60: $60 > 50 \rightarrow$ go right to 70; $60 < 70 \rightarrow$ go left to 60; found (path $50 \rightarrow 70 \rightarrow 60$). 65: $50 \rightarrow 70 \rightarrow 60$; $65 > 60$ but 60 has no right child, so the search fails.

(b) Successor of 40 = smallest key greater than 40 = 50 (40 has no right subtree, so it is the nearest ancestor for which 40 lies in the left subtree). Predecessor of 60 = largest key smaller than 60 = 50.

EXAMPLE

Insert 10, 20, 30, 40 into an empty BST. What is the search cost for 40?

Each key is larger than all earlier keys, so each goes to the right: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40$, a chain of height 3. Searching 40 checks all 4 nodes: $O(n)$. This is the worst case.

Exam tip: the unit says "the complexity of BST for search, insert and delete is $O(n)$ ". That is the worst case. Write " $O(h)$: $O(\log n)$ on average, $O(n)$ in the worst case" to get full marks.

Scapegoat trees and α -weight balance

Scapegoat tree: a self-balancing binary search tree that gives $O(\log n)$ worst-case search and $O(\log n)$ amortized insert and delete, by rebuilding the subtree of an unbalanced "scapegoat" node.

α -weight-balanced node: a node x with $\text{size}(\text{left}(x)) \leq \alpha \cdot \text{size}(x)$ and $\text{size}(\text{right}(x)) \leq \alpha \cdot \text{size}(x)$, where $0.5 \leq \alpha < 1$.

Scapegoat tree: balance തെറ്റിയ ഒരു "scapegoat" node-ന്റെ subtree മുഴുവൻ പുനർനിർമ്മിച്ച് balance നിലനിർത്തുന്ന self-balancing binary search tree; search-ന് worst case $O(\log n)$, insert-നും delete-നും amortized $O(\log n)$.

α -weight-balanced node: $\text{size}(\text{left}(x)) \leq \alpha \cdot \text{size}(x)$, $\text{size}(\text{right}(x)) \leq \alpha \cdot \text{size}(x)$ എന്നീ രണ്ട് നിബന്ധനകളും പാലിക്കുന്ന node x ($0.5 \leq \alpha < 1$).

$\text{size}(x)$ is the number of keys in the subtree rooted at x , including x itself. A node is α -weight-balanced when neither child holds "too much" of its subtree. With $\alpha = 0.5$ both sides must be exactly equal (too strict); with α close to 1 almost anything is allowed. A common choice is $\alpha = 2/3$: neither child may hold more than two-thirds of the subtree. Unlike AVL or red-black trees, a scapegoat tree stores **no balance information in the nodes** (no heights, no colours); it only keeps two counters for the whole tree, n (current size) and maxN .

മലയാളത്തിൽ

ഒരു തുലാസ് ഓർക്കുക: ഒരു വശത്ത് ഭാരം വളരെ കൂടിയാൽ അത് ചരിയും. Node-ന്റെ രണ്ട് കുട്ടികളിൽ ഒരാൾ മാത്രം subtree-യുടെ α ഭാഗത്തിലധികം keys കൈവശം വെച്ചാൽ ആ node balance തെറ്റിയതാണ്. Tree-യുടെ ചരിവിന് "കുറ്റക്കാരൻ" ആയി ഒരു node-നെ കണ്ടെത്തി (scapegoat = ബലിയാട്), അതിന്റെ subtree മാത്രം വീണ്ടും നേരെയാക്കുന്നു. AVL tree പോലെ ഓരോ node-ലും height സൂക്ഷിക്കേണ്ടതില്ല എന്നതാണ് ഇതിന്റെ സൗകര്യം.

$$\text{size}(x) = \text{size}(\text{left}(x)) + \text{size}(\text{right}(x)) + 1$$

x is α -weight-balanced $\Leftrightarrow \text{size}(\text{left}(x)) \leq \alpha \cdot \text{size}(x)$ and $\text{size}(\text{right}(x)) \leq \alpha \cdot \text{size}(x)$, $0.5 \leq \alpha < 1$

Size of a parent while walking up: $\text{size}(x_{i+1}) = \text{size}(x_i) + \text{size}(\text{brother}(x_i)) + 1$

Height guarantee: $\text{height} \leq \text{about } \log_{1/\alpha} n$ (for $\alpha = 2/3$, $\log_{1.5} n$)

EXAMPLE

Take $\alpha = 2/3$. Is each node α -weight-balanced? (a) the tree $4(2(1,3), 6(5,7))$; (b) the chain $1 \rightarrow 2 \rightarrow 3$ (each key the right child of the previous one).

(a) Root 4: size = 7, left size 3, right size 3; limit $\alpha \cdot 7 = 4.67$, and $3 \leq 4.67$ on both sides $\checkmark$. Node 2: size 3, children of size 1 each $\leq 2 \checkmark$; node 6 likewise $\checkmark$; leaves trivially $\checkmark$. The whole tree is α -weight-balanced.

(b) Root 1: size 3, right size 2, limit $\alpha \cdot 3 = 2$, $2 \leq 2 \checkmark$. Node 2: size 2, right size 1, limit $1.33 \checkmark$. Surprisingly still balanced for $\alpha = 2/3$. Add 4 below 3: root size 4, right size 3 $> 2.67 \times$, so node 1 is not α -weight-balanced.

Exam tip: IGNOU writes "scapegoat tries" throughout. The correct name is scapegoat **trees**; they are ordinary BSTs, not tries. Also, the unit's line "balanced when the weight of the left nodes matches the weight of the right nodes" is only informal: the real condition is the α inequality above, which allows the two sides to differ.

Operations on scapegoat trees

Scapegoat node: an ancestor of a newly inserted, too-deep node that is not α -weight-balanced; its subtree is rebuilt into a perfectly balanced BST.

Scapegoat node: പുതുതായി insert ചെയ്ത, അമിത ആഴത്തിലുള്ള node-ന്റെ α -weight-balanced അല്ലാത്ത ഒരു ancestor; അതിന്റെ subtree പൂർണ്ണമായി balanced BST ആയി പുനർനിർമ്മിക്കുന്നു.

Search is exactly BST search. Because the height stays $O(\log n)$, it is $O(\log n)$ in the worst case.

Insert Y: (1) create node x and insert it the normal BST way, noting its depth d . (2) If $d > \log_{1/\alpha} n$ (n = size after inserting), walk up from x . At each ancestor compute its size using $\text{size}(\text{parent}) = \text{size}(\text{child}) + \text{size}(\text{brother}) + 1$, and stop at the first ancestor that is not α -weight-balanced, i.e. $\text{size}(\text{child}) > \alpha \cdot \text{size}(\text{ancestor})$. That ancestor is the scapegoat. (3) Rebuild the scapegoat's subtree: list its keys by inorder traversal (already sorted), make the middle key the root, and build the two halves recursively. Rebuilding a subtree of size s costs $O(s)$, but it happens rarely, so the amortized cost of insert is $O(\log n)$.

Delete x (BST delete): (1) if x is a leaf, remove it; (2) if x has one child, replace x by that child; (3) if x has two children, replace x by the smallest key in its right subtree (the inorder successor). Then decrease n. The tree also keeps maxN, the largest n since the last full rebuild; if $n < \alpha \cdot \text{maxN}$, rebuild the whole tree and set $\text{maxN} = n$.

മലയാളത്തിൽ

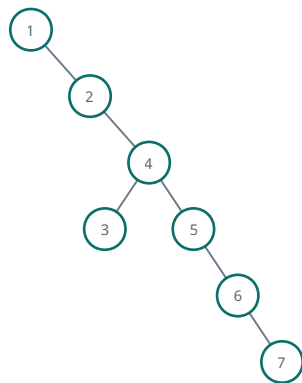
Insert സാധാരണ BST പോലെ തന്നെ. പുതിയ node വളരെ ആഴത്തിലാണ് ($\text{depth} > \log_{1/\alpha} n$) എങ്കിൽ മാത്രം അതിൽ നിന്ന് മുകളിലേക്ക് parent വഴി നടന്നു ഓരോ ancestor-ന്റെയും size കണക്കാക്കുന്നു. ആദ്യം കാണുന്ന balance തെറ്റിയ ancestor ആണ് scapegoat. അതിന്റെ subtree-യിലെ keys inorder ക്രമത്തിൽ എടുത്ത്, നടുവിലെ key root ആക്കി വീണ്ടും കെട്ടിപ്പടുക്കുന്നു, അലമാര മുഴുവൻ ഒരമിച്ച് അടുക്കി വെക്കുന്നതുപോലെ. Delete-ൽ ധാരാളം node-കൾ പോയാൽ ($n < \alpha \cdot \text{maxN}$) tree മുഴുവൻ ഒറ്റയടിക്ക് rebuild ചെയ്യുന്നു.

Rebuild trigger on insert: $\text{depth}(x) > \log_{1/\alpha} n$

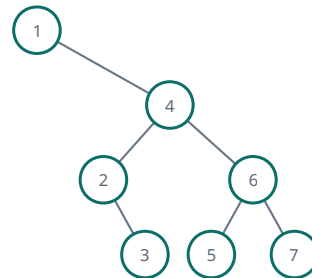
Scapegoat: first ancestor a (going up) with $\text{size}(\text{child on path}) > \alpha \cdot \text{size}(a)$

Rebuild trigger on delete: $n < \alpha \cdot \text{maxN} \Rightarrow \text{rebuild whole tree, maxN} = n$

Search $O(\log n)$ worst case; insert, delete $O(\log n)$ amortized



After inserting 7 (depth 5).
Scapegoat = 2.



Subtree of 2 rebuilt; maximum
depth is now 3.

EXAMPLE (INSERT WITH REBUILD, $\alpha = 2/3$)

Keys 1, 2, 4, 3, 5, 6 were inserted into a scapegoat tree (no rebuild was needed). Insert 7.

Step 1: BST insert: $7 > 1, 2, 4, 5, 6$, so 7 becomes the right child of 6 at depth 5 (left figure).

Step 2: $n = 7$ and $\log_{1.5} 7 = \ln 7 / \ln 1.5 = 1.946 / 0.405 \approx 4.80$. Depth 5 $>$ 4.80, so we must find a scapegoat. Walk up:

Ancestor	size(child on path)	size(ancestor) = child + brother + 1	α -size	child $>$ α -size?
6	1 (node 7)	$1 + 0 + 1 = 2$	1.33	No
5	2 (node 6)	$2 + 0 + 1 = 3$	2.00	No
4	3 (node 5)	$3 + 1 + 1 = 5$	3.33	No
2	5 (node 4)	$5 + 0 + 1 = 6$	4.00	Yes $\rightarrow$ scapegoat

Step 3: rebuild the subtree of 2. Inorder list: 2, 3, 4, 5, 6, 7. Middle (index $\lfloor (0+5)/2 \rfloor = 2$) is 4 $\rightarrow$ root. Left half 2, 3 $\rightarrow$ root 2 with right child 3. Right half 5, 6, 7 $\rightarrow$ root 6 with children 5 and 7. Attach the new subtree as the right child of 1 (right figure). Maximum depth drops from 5 to $3 \leq 4.80 \checkmark$.

EXAMPLE (DELETE)

From the rebuilt tree $1(-, 4(2(-,3), 6(5,7)))$, delete 4. Suppose $\max N = 7$.

4 has two children, so replace it by the smallest key in its right subtree, 5 (leftmost node of 6's subtree).

Tree: $1(-, 5(2(-,3), 6(-,7)))$. Now $n = 6$ and $\alpha \cdot \max N = 2/3 \times 7 = 4.67$. Since $6 \geq 4.67$, no full rebuild is needed.

Exam tip: the unit's insert steps omit the depth test and the rebuild step; mention both (walk up only when depth $>$ $\log_{1/\alpha} n$, then rebuild the scapegoat's subtree as a perfectly balanced BST). Its delete steps omit the maxN rule. Note also that after the rebuild above, root 1 is still not α -weight-balanced (right size $6 > 4.67$). That is allowed: the scapegoat method guarantees a small height, not that every node is weight-balanced.

Tries

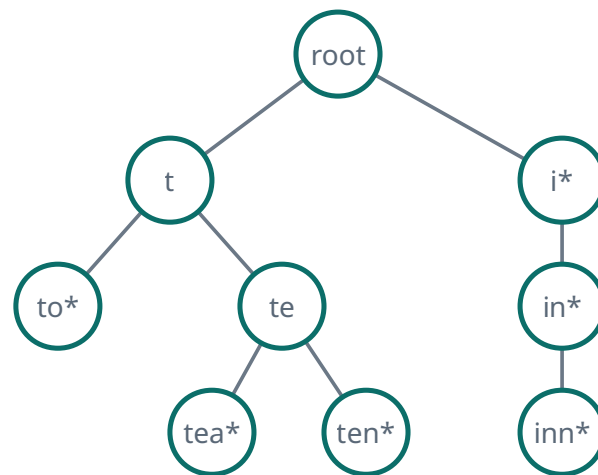
Trie (prefix tree): a tree-based data structure for storing and searching string keys, in which each edge is labelled by a character and all descendants of a node share the common prefix spelt out by the path from the root.

Trie (prefix tree): string keys സൂക്ഷിക്കാനും തിരയാനും ഉള്ള tree data structure; ഓരോ edge-ഉം ഒരു character ആണ്, ഒരു node-ന്റെ എല്ലാ descendants-നും root മുതൽ ആ node വരെയുള്ള path നൽകുന്ന common prefix ഉണ്ടാകും.

A key is not stored inside a node; it is given by the **position** of the node, i.e. by the characters on the path from the root. Nodes where a key ends are marked. Main characteristics (IGNOU): (1) the children of a node share a common prefix; (2) the key is denoted by the position of the node; (3) strings can be searched by a depth-first walk from the root. To search, insert or delete a string of length L we follow L edges, so the time is $O(L)$, independent of how many keys are stored. Tries are used for dictionaries, autocomplete, spell checkers and search engines; bitwise tries are used for integer look-ups such as ID numbers or account numbers.

മലയാളത്തിൽ

Mobile-ലെ contact list-ൽ "Ra" എന്ന് type ചെയ്യുമ്പോൾ Rahul, Rajan, Ramesh എല്ലാം കാണിക്കുന്നത് ഓർക്കുക. ഇവർക്കെല്ലാം "Ra" എന്ന common prefix ഉണ്ട്; trie-യിൽ അവർ ഒരേ branch-ൽ ആണ്. ഓരോ character-ഉം ഒരു പടി; വാക്കിന്റെ നീളം L ആണെങ്കിൽ L പടി മാത്രം നടന്നാൽ മതി, tree-യിൽ എത്ര വാക്കുകൾ ഉണ്ടെങ്കിലും.



Trie for {to, tea, ten, i, in, inn}. Each node shows its prefix; * marks the end of a stored key.

Search / insert / delete a key of length L : $O(L)$

Number of nodes $\leq 1 + \text{total number of characters in all keys}$

EXAMPLE

Build the trie for to, tea, ten, i, in, inn. How many nodes does it have? Is "te" a key? Is "tent" a key?

Insert "to": root $\rightarrow$ t $\rightarrow$ o. "tea": t exists $\rightarrow$ e $\rightarrow$ a. "ten": t, e exist $\rightarrow$ n. "i": root $\rightarrow$ i. "in": i exists $\rightarrow$ n. "inn": i, n exist $\rightarrow$ n. Distinct prefixes: t, to, te, tea, ten, i, in, inn = 8 nodes + root = 9 nodes.

"te": the path root $\rightarrow$ t $\rightarrow$ e exists but node "te" is not marked as a key end, so "te" is **not** a key. "tent": after "ten" there is no edge labelled t, so the search fails.

Exam tip: the unit says the average time for trie search, insert and delete is " $O(n)$ ". Here n means the **length of the key**, not the number of keys. Write $O(L)$ (or $O(m)$) with L = key length and say so; that is why tries are fast even for millions of stored words.

Binary tries

Binary trie: a trie over the alphabet {0, 1} in which keys are bit strings, the i -th bit decides the branch at level i (0 = left, 1 = right), and only the leaf nodes hold keys.

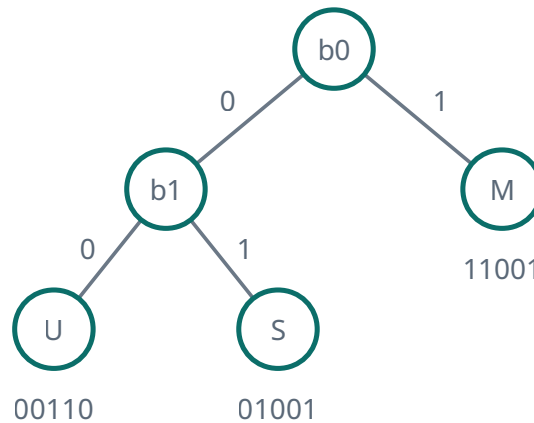
Binary trie: keys bit strings ആയ, {0, 1} alphabet ഉള്ള trie; level i -ൽ i -ാമത്തെ bit ആണ് വഴി തീരുമാനിക്കുന്നത് (0 = left, 1 = right), keys leaf nodes-ൽ മാത്രം.

Integers (or the bit codes of characters) are written in binary and the bits are used to walk down the tree. Internal nodes are only branch points; a key sits in a leaf. A new internal node is created only where two keys must be told apart, so a leaf can sit higher than the full bit length. Because 0 goes left and 1 goes right, for every node the keys in the left subtree are smaller than those in the right subtree.

Search k : start at the root with $i = 0$; if bit i of k is 0 go left, else go right; at a leaf, succeed only if the whole stored key equals k ; otherwise increase i and repeat. **Insert k :** search for k ; if the search fails at an internal node with an empty child, link a new leaf there; if it fails at a leaf holding another key, create new internal node(s) until the first bit where the two keys differ, and hang both leaves below it. **Delete k :** search for k and delete its leaf; walk back up the search path deleting nodes until reaching a node u that still has a child not on the path of k .

മലയാളത്തിൽ

വഴി ചോറിച്ച് വീട് കണ്ടെത്തുന്നതുപോലെ: ഓരോ കവലയിലും bit 0 ആണെങ്കിൽ ഇടത്തോട്ട്, 1 ആണെങ്കിൽ വലത്തോട്ട്. രണ്ട് keys-നെ വേർതിരിക്കേണ്ട സ്ഥലത്ത് മാത്രമേ പുതിയ കവല (internal node) ഉണ്ടാക്കൂ. Leaf-ൽ എത്തിയാൽ അവിടെയുള്ള key മുഴുവനായി നമ്മുടെ key-യുമായി ഒത്തുനോക്കണം, അല്ലെങ്കിൽ തെറ്റായ ഉത്തരം കിട്ടും.



Binary trie for $S = 01001$, $M = 11001$, $U = 00110$. b_0 , b_1 = internal nodes that test bit 0, bit 1.

EXAMPLE (THE UNIT'S FIGURE 5)

Insert $S = 01001$, $M = 11001$, $U = 00110$ (in this order) into an empty binary trie.

S : bit 0 = 0, the root's left child is empty → link leaf S there. M : bit 0 = 1, right child empty → link leaf M .

U : bit 0 = 0 → reach leaf S (search fails at a leaf). Compare $U = 00110$ with $S = 01001$: bit 0 same (0), bit 1 differs (U has 0, S has 1). Create a new internal node testing bit 1 in place of S : U goes left, S goes right.

Order check: $U = 6$, $S = 9$, $M = 25$ in decimal. Reading leaves left to right gives $U, S, M = 6, 9, 25$, sorted ✓.

EXAMPLE

In the trie above, (a) insert $X = 01011$, (b) search 01100.

(a) Bit 0 = 0 → b_1 ; bit 1 = 1 → leaf $S = 01001$. Compare 01011 with 01001: bits 0, 1, 2 agree (0, 1, 0), bit 3 differs (X has 1, S has 0). Replace leaf S by a node testing bit 2 whose only child (bit 2 = 0, left) is a node testing bit 3; under it S goes left (0) and X goes right (1). $X = 11$ lies between $S = 9$ and $M = 25$ ✓.

(b) Bit 0 = 0 → b_1 ; bit 1 = 1 → leaf S . The leaf's key 01001 ≠ 01100, so the search fails. The final full comparison is essential.

EXAMPLE (DELETE)

Delete S from the trie of U, S, M (figure).

Search S : $b_0 \rightarrow b_1 \rightarrow$ leaf S ; delete the leaf. Walk back: b_1 still has another child (U) not on S 's path, so we stop and keep b_1 . (Many books then collapse b_1 , which now has one child, and move U up to b_1 's place; the tree still works either way.)

X-fast tries

X-fast trie: a bitwise trie over the bounded integer universe $\{0, \dots, M - 1\}$ whose every level is stored in a hash table, whose leaves form a sorted doubly linked list, and whose one-child nodes keep descendant pointers; it gives $O(1)$ find and $O(\log \log M)$ successor/predecessor.

Descendant pointer: in a node without a left child, a pointer to the smallest leaf in its right subtree; in a node without a right child, a pointer to the largest leaf in its left subtree.

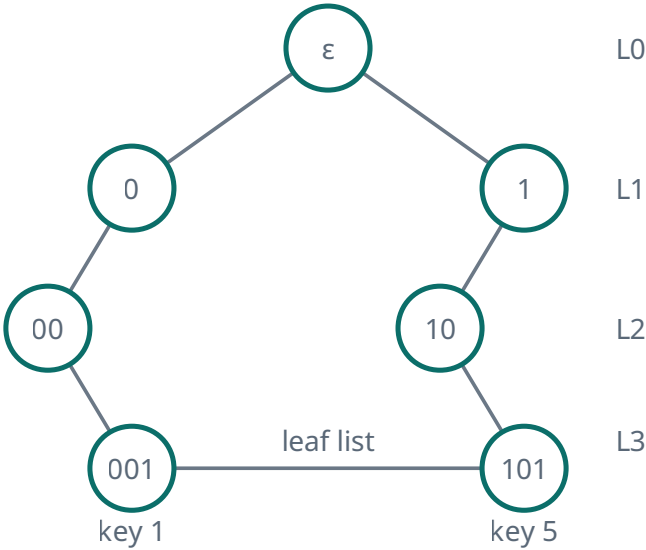
X-fast trie: $\{0, \dots, M - 1\}$ എന്ന പരിമിത integer domain-നുള്ള bitwise trie; ഓരോ level-ഉം ഒരു hash table, leaves ഒരു sorted doubly linked list, ഒരു child മാത്രമുള്ള node-കളിൽ descendant pointer; find $O(1)$, successor/predecessor $O(\log \log M)$.

Descendant pointer: left child ഇല്ലാത്ത node-ൽ അതിന്റെ right subtree-യിലെ ഏറ്റവും ചെറിയ leaf-ലേക്കും, right child ഇല്ലാത്ത node-ൽ left subtree-യിലെ ഏറ്റവും വലിയ leaf-ലേക്കും ഉള്ള pointer.

Every key is written with $w = \log_2 M$ bits, so the trie has $w + 1$ levels and all keys sit at the bottom level. A node at level i stands for an i -bit prefix; its left child appends 0 and its right child appends 1. An internal node exists only if its subtree holds at least one key. Level i is a hash table of the prefixes present at that level, so "does prefix p exist?" is an $O(1)$ look-up. The leaves are linked in sorted order, so once one neighbour of a query is found, the other is one step away. Each key stores all w of its prefixes, so space is $O(n \log M)$.

മലയാളത്തിൽ

ഒരു വലിയ office-ലെ file ഷെൽഫ് ഓർക്കുക: ഓരോ നിലയിലും (level) ഏതൊക്കെ prefix-കൾ ഉണ്ട് എന്ന ഒരു register (hash table) ഉണ്ട്. ഒരു prefix ഉണ്ടോ എന്ന് അറിയാൻ ആ register ഒന്ന് നോക്കിയാൽ മതി, $O(1)$. Levels $\log M$ എണ്ണം മാത്രമേ ഉള്ളൂ; അവയിൽ binary search ചെയ്യുന്നതുകൊണ്ട് successor കണ്ടെത്താൻ $O(\log \log M)$ സമയം മതി.



Four-level X-fast trie, $M = 8$ (3 bits), storing 1 = 001 and 5 = 101. The bottom line is the leaf linked list.

Level (hash table)	Prefixes stored
L0	ϵ (root)
L1	0, 1
L2	00, 10
L3 (leaves)	001 (1), 101 (5); linked 1 $\leftrightarrow$ 5

Node	Left pointer	Right pointer
ϵ	child 0	child 1
0	child 00	no child $\rightarrow$ descendant pointer to largest leaf in left subtree = 1
1	child 10	no child $\rightarrow$ descendant pointer to largest leaf in left subtree = 5
00	no child $\rightarrow$ descendant pointer to smallest leaf in right subtree = 1	leaf 001
10	no child $\rightarrow$ descendant pointer to smallest leaf in right subtree = 5	leaf 101

$w = \log_2 M$ bits per key; $w + 1$ levels

Find: $O(1)$ · Successor / predecessor: $O(\log \log M)$

Insert / delete: $O(\log M)$ · Space: $O(n \log M)$

Exam tip: the unit says the trie stores "0 (001) and 5 (101)". 001 is the binary code of 1, not 0 (0 would be 000). The keys in the figure are 1 and 5. The unit also calls this "Figure 5" in the text; it is Figure 6.

Operations on X-fast tries

Lowest ancestor of k : the deepest trie node whose prefix equals a prefix of k 's bit string; it is found by binary search on the prefix length, using the level hash tables.

k -യുടെ lowest ancestor: k -യുടെ bit string-ന്റെ ഒരു prefix-ന് തുല്യമായ prefix ഉള്ള ഏറ്റവും ആഴത്തിലുള്ള trie node; level hash tables ഉപയോഗിച്ച് prefix length-ൽ binary search ചെയ്താണ് ഇത് കണ്ടെത്തുന്നത്.

Find k : look k up in the hash table of the leaf level: $O(1)$.

Successor / predecessor of k : (1) binary-search on the prefix length ℓ between 0 and w : if the first ℓ bits of k are in the level- ℓ hash table, try longer prefixes, else shorter ones. This gives the lowest ancestor in $O(\log w) = O(\log \log M)$ look-ups. (2) The lowest ancestor is missing the child that k would go to, so that side holds a descendant pointer. If the right child is missing, the pointer gives the largest leaf on the left = predecessor of k ; if the left child is missing, it gives the smallest leaf on the right = successor of k . (3) The other neighbour is the next (for successor) or previous (for predecessor) leaf in the linked list.

Insert k : find the predecessor and successor of k ; create a new leaf between them in the linked list (and in the leaf hash table); walk from the root to the leaf creating missing internal nodes (adding them to their level tables) and fixing descendant pointers: $O(\log M)$. **Delete k :** remove k from the leaf hash table; unlink the leaf, joining its predecessor and successor; walk up deleting internal nodes whose subtree contained only k , and update descendant pointers: $O(\log M)$.

മലയാളത്തിൽ

Successor കണ്ടെത്താൻ ആദ്യം k -യുടെ ഏറ്റവും നീളമുള്ള ഏത് prefix ആണ് trie-യിൽ ഉള്ളത് എന്ന് കണ്ടെത്തണം. എല്ലാ level-ഉം ഓരോന്നായി നോക്കാതെ, നടുവിലെ level ആദ്യം നോക്കുന്നു: prefix ഉണ്ടെങ്കിൽ താഴോട്ട്, ഇല്ലെങ്കിൽ മുകളിലോട്ട്, ഒരു binary search. അവിടെ നിന്ന് descendant pointer വഴി ഒരു അയൽക്കാരനെയും (neighbour) leaf list വഴി മറ്റേ അയൽക്കാരനെയും കിട്ടും.

EXAMPLE (SUCCESSOR AND PREDECESSOR OF 3)

Trie of the previous topic (keys 1 and 5, $w = 3$). Find $\text{succ}(3)$ and $\text{pred}(3)$.

$3 = 011$. Binary search on $\ell \in [0, 3]$: try $\ell = 2$, prefix "01": not in $L_2 \rightarrow$ shorter. Try $\ell = 1$, prefix "0": in $L_1 \rightarrow$ lowest ancestor is node "0".

The next bit of 3 is 1, and node "0" has no right child, so its right pointer is a descendant pointer to the largest leaf in its left subtree: 1. So $\text{pred}(3) = 1$. The next leaf after 1 in the list is 5, so $\text{succ}(3) = 5$.

EXAMPLE (A QUERY THAT USES A LEFT DESCENDANT POINTER)

Same trie. Find $\text{succ}(0)$ and $\text{succ}(6)$.

$0 = 000$: $\ell = 2$, "00" is in $L_2 \rightarrow$ longer; $\ell = 3$, "000" is not in $L_3 \rightarrow$ lowest ancestor "00". The next bit is 0 and "00" has no left child, so its descendant pointer gives the smallest leaf on the right, 1: $\text{succ}(0) = 1$ (no leaf before it, so $\text{pred}(0)$ does not exist).

$6 = 110$: "11" not in L_2 ; "1" in $L_1 \rightarrow$ lowest ancestor "1". Next bit 1, no right child $\rightarrow$ descendant pointer = 5 = $\text{pred}(6)$. The leaf after 5 does not exist, so $\text{succ}(6)$ does not exist.

EXAMPLE (INSERT 3, THEN DELETE 1)

Insert 3 (011) into the trie of {1, 5}.

$\text{pred}(3) = 1$, $\text{succ}(3) = 5$ (above). New leaf 011 goes between them: list $1 \leftrightarrow 3 \leftrightarrow 5$. Walking down $0 \rightarrow 01 \rightarrow 011$: "0" exists, "01" is new $\rightarrow$ add to L_2 . Tables: $L_1 = \{0, 1\}$, $L_2 = \{00, 01, 10\}$, $L_3 = \{001, 011, 101\}$. Pointers: "0" now has both children (no descendant pointer); the new node "01" has no left child, so it points to the smallest leaf on its right, 3.

Now delete 1 (001). Remove 001 from L_3 ; link its neighbours (list becomes $3 \leftrightarrow 5$). Node "00" held only 1, so delete it from L_2 . Tables: $L_1 = \{0, 1\}$, $L_2 = \{01, 10\}$, $L_3 = \{011, 101\}$. Node "0" now has no left child, so it gets a descendant pointer to the smallest leaf on its right, 3.

Exam tip: explain why successor is $O(\log \log M)$: there are only $w = \log M$ levels, and a binary search over w levels takes $O(\log w) = O(\log \log M)$ hash look-ups, each $O(1)$. Insert/delete must touch all w ancestors, hence $O(\log M)$.

Y-fast tries

Y-fast trie: a two-level structure for integers from a bounded universe $\{0, \dots, M - 1\}$: keys are split into groups of $\Theta(\log M)$ consecutive keys, each group stored in a balanced BST, and one representative per group stored in an X-fast trie. It uses $O(n)$ space.

Y-fast trie: $\{0, \dots, M - 1\}$ domain-ലെ integers-ന് ഉള്ള രണ്ട് തട്ടുള്ള structure: keys-നെ $\Theta(\log M)$ വലുപ്പമുള്ള തുടർച്ചയായ groups ആക്കി ഓരോന്നും balanced BST-യിൽ സൂക്ഷിക്കുന്നു, ഓരോ group-ന്റെയും ഒരു representative X-fast trie-യിൽ; space $O(n)$.

An X-fast trie is fast but uses $O(n \log M)$ space because every key stores all its prefixes. The Y-fast trie fixes this. The sorted keys are cut into consecutive groups of size between $(\log M)/4$ and $2 \log M$. Each group is a balanced BST (so searching inside a group of size $O(\log M)$ costs $O(\log \log M)$). One **representative** r per group (for example its smallest key) goes into the X-fast trie, and each X-fast leaf points to its group's BST. Since the X-fast trie holds only about $n / \log M$ representatives, its space is $O((n / \log M) \cdot \log M) = O(n)$.

മലയാളത്തിൽ

ഒരു വലിയ school ഓർക്കുക: ഓരോ കുട്ടിയുടെയും വിവരം principal-ന്റെ register-ൽ (X-fast trie) എഴുതുന്നതിനു പകരം, കുട്ടികളെ ക്ലാസുകളായി (groups, balanced BST) തിരിച്ച് ഓരോ ക്ലാസിന്റെയും class leader-നെ (representative) മാത്രം register-ൽ എഴുതുന്നു. ആദ്യം ശരിയായ ക്ലാസ് കണ്ടെത്തുക, പിന്നെ ക്ലാസിനുള്ളിൽ തിരയുക. Register ചെറുതായതുകൊണ്ട് memory ലാഭം; വേഗം ഏതാണ്ട് അതുതന്നെ.

X-fast trie (top)	:	representatives	0	5	12
Balanced BSTs	:	groups	{0,1,2,3,4}	{5,6,7,8}	{12,14}

Operation	BST (balanced)	X-fast trie	Y-fast trie
Find	$O(\log n)$	$O(1)$	$O(\log \log M)$
Successor / predecessor	$O(\log n)$	$O(\log \log M)$	$O(\log \log M)$
Insert / delete	$O(\log n)$	$O(\log M)$	$O(\log \log M)$ amortized
Space	$O(n)$	$O(n \log M)$	$O(n)$

Exam tip: the unit copies the X-fast bounds to the Y-fast trie ("find $O(1)$ "; insert and delete $O(\log M)$ "). That is wrong. For a Y-fast trie, find is $O(\log \log M)$ (the key must be searched inside a BST of size $O(\log M)$), and insert/delete are $O(\log \log M)$ amortized. The gain over X-fast is space: $O(n)$ instead of $O(n \log M)$.

Operations on Y-fast tries

Representative: one key chosen from each group (BST) of a Y-fast trie and stored in the X-fast trie; it identifies the group and points to its BST.

Representative: Y-fast trie-യിലെ ഓരോ group-ൽ (BST) നിന്നും തിരഞ്ഞെടുത്ത് X-fast trie-യിൽ സൂക്ഷിക്കുന്ന ഒരു key; അത് ആ group-നെ തിരിച്ചറിയുകയും അതിന്റെ BST-യിലേക്ക് point ചെയ്യുകയും ചെയ്യുന്നു.

Find k: in the X-fast trie find $r_1 =$ the largest representative $\leq k$ (predecessor side) and $r =$ the smallest representative $> k$ (successor side); then search for k in the BSTs of r_1 and r . **Successor of k:** find these two representatives in the X-fast trie and search their two BSTs for the smallest key $> k$. Predecessor is symmetric. Each step is $O(\log \log M)$.

Insert k: find the group where k belongs (via its successor/predecessor representative) and insert k into that BST. If the BST now has more than $2 \log M$ elements, remove its representative from the X-fast trie, split the BST into two halves, and insert the two new representatives. **Delete k:** delete k from its BST. If the BST drops below $(\log M)/4$ elements, merge it with a neighbouring BST (predecessor or successor group), remove both old representatives from the X-fast trie and insert the representative of the merged tree; if the merged tree has more than $2 \log M$ elements, split it as in insert. X-fast updates cost $O(\log M)$ but happen only once every $\Theta(\log M)$ operations, so the amortized cost is $O(\log \log M)$.

മലയാളത്തിൽ

Insert ചെയ്യുമ്പോൾ ഒരു ക്ലാസിൽ കുട്ടികൾ അമിതമായാൽ ($2 \log M$ -ൽ കൂടുതൽ) ക്ലാസ് രണ്ടായി പിരിക്കുന്നു, രണ്ട് പുതിയ class leaders register-ൽ വരുന്നു. Delete ചെയ്ത് ക്ലാസ് വളരെ ചെറുതായാൽ $((\log M)/4$ -ൽ കുറവ്) അടുത്ത ക്ലാസുമായി ലയിപ്പിക്കുന്നു. ഈ register മാറ്റങ്ങൾ വല്ലപ്പോഴും മാത്രം നടക്കുന്നതുകൊണ്ട് ശരാശരി ചെലവ് (amortized) $O(\log \log M)$ മാത്രം.

Group size kept between $(\log M)/4$ and $2 \log M$

Insert: group size $> 2 \log M \Rightarrow$ split into two, update representatives

Delete: group size $< (\log M)/4 \Rightarrow$ merge with a neighbour (then split if $> 2 \log M$)

EXAMPLE (INSERT WITH A SPLIT)

$M = 16$, so $\log M = 4$: groups may hold between $4/4 = 1$ and $2 \times 4 = 8$ keys. Groups (representative = smallest key): $A = \{0, 1, 2, 3, 5, 6, 7, 8\}$ with $r = 0$, $B = \{12, 14\}$ with $r = 12$. Insert 4.

Largest representative ≤ 4 is 0, so 4 goes into A: $A = \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$, 9 keys > 8 . Split: remove representative 0 from the X-fast trie, split into $A1 = \{0, 1, 2, 3, 4\}$ and $A2 = \{5, 6, 7, 8\}$, insert representatives 0 and 5. The X-fast trie now holds $\{0, 5, 12\}$ (the picture in the previous topic).

EXAMPLE (FIND AND SUCCESSOR)

With groups $\{0, \dots, 4\}$, $\{5, 6, 7, 8\}$, $\{12, 14\}$, find 10 and $\text{succ}(10)$.

In the X-fast trie: predecessor representative of 10 is 5, successor representative is 12. Search $\text{BST}(5) = \{5, 6, 7, 8\}$: 10 not there. Search $\text{BST}(12) = \{12, 14\}$: not there. So 10 is not a key. Smallest key > 10 in these two BSTs: none in $\text{BST}(5)$, 12 in $\text{BST}(12)$, so $\text{succ}(10) = 12$.

EXAMPLE (DELETE WITH A MERGE)

Continue: delete 12, then 14.

Delete 12: $B = \{14\}$, size 1, not below 1, so nothing else changes (the representative can be updated to 14).

Delete 14: B is empty, size $0 < 1 \rightarrow$ merge with the predecessor group $\{5, 6, 7, 8\}$: remove B 's representative from the X-fast trie; the merged group $\{5, 6, 7, 8\}$ keeps representative 5 and has $4 \leq 8$ keys, so no split is needed.

Exam tip: a standard 5-mark question asks "How does a Y-fast trie improve on an X-fast trie?" Answer: same $O(\log \log M)$ successor/predecessor, but space drops from $O(n \log M)$ to $O(n)$, because only one representative per group of $\Theta(\log M)$ keys goes into the X-fast trie; insert/delete become $O(\log \log M)$ amortized.

Quick recap

- BST: left keys $<$ node $<$ right keys. Operations cost $O(h)$: $O(\log n)$ when balanced, $O(n)$ in the worst case (skewed tree).
- Successor of k = smallest key $> k$; predecessor = largest key $< k$.
- Scapegoat tree: self-balancing BST with no per-node balance data; $O(\log n)$ worst-case search, $O(\log n)$ amortized insert/delete.
- α -weight-balanced: $\text{size}(\text{left}(x)) \leq \alpha \cdot \text{size}(x)$ and $\text{size}(\text{right}(x)) \leq \alpha \cdot \text{size}(x)$, $0.5 \leq \alpha < 1$; $\text{size}(x)$ counts x and all its descendants.
- Insert: if the new node's depth $> \log_{1/\alpha} n$, walk up ($\text{size}(\text{parent}) = \text{size}(x) + \text{size}(\text{brother}) + 1$) to the first unbalanced ancestor (the scapegoat) and rebuild its subtree as a perfectly balanced BST.
- Delete: normal BST delete; rebuild the whole tree when $n < \alpha \cdot \max N$.

- Trie: characters on edges, key = position of node, children share a common prefix; search/insert/delete in $O(L)$, L = key length.
- Binary trie: bits as keys, $0 \rightarrow$ left, $1 \rightarrow$ right, keys only in leaves; new internal nodes only where two keys first differ.
- X-fast trie: bounded universe $0 \dots M - 1$, each level a hash table, leaves in a sorted linked list, descendant pointers in one-child nodes.
- X-fast: find $O(1)$, successor/predecessor $O(\log \log M)$ (binary search over the $\log M$ levels), insert/delete $O(\log M)$, space $O(n \log M)$.
- Y-fast trie: groups of $\Theta(\log M)$ keys in balanced BSTs + representatives in an X-fast trie; space $O(n)$.
- Y-fast: find, successor, predecessor $O(\log \log M)$; insert/delete $O(\log \log M)$ amortized; split a group above $2 \log M$, merge below $(\log M)/4$.

Check yourself

Q. Fill in the blanks (CYP 1): (1) Scapegoat trees are ____ BSTs. (2) For inserting a node we find the ____ node. (3) The worst-case performance of scapegoat search is _____. (4) In a BST the left node value is ____ than the right node value. (5) The complexity of BST search, insert and delete is _____.

(1) self-balancing (2) scapegoat (3) $O(\log n)$ (4) lesser (5) $O(n)$ in the worst case ($O(h)$ in general, $O(\log n)$ for a balanced tree).

Q. Fill in the blanks (CYP 2): (1) The children of a node in a trie share a _____. (2) Average time of trie search/insert/delete is _____. (3) Binary tries use _____ as keys. (4) X-fast tries store integers from a _____. (5) Each level of an X-fast trie is implemented as a _____. (6) Y-fast tries are an improved version of _____.

(1) common prefix (2) $O(n)$ where n is the key length, better written $O(L)$ (3) bits (4) bounded domain (5) hash table (6) X-fast tries.

Q. With $\alpha = 0.6$, is a node with size 10 whose left subtree has 7 nodes α -weight-balanced?

Limit = $\alpha \cdot \text{size} = 0.6 \times 10 = 6$. The left subtree has $7 > 6$, so the node is **not** α -weight-balanced. (Its right subtree has $10 - 7 - 1 = 2 \leq 6$, but both conditions must hold.)

Q. Insert 4 into the scapegoat tree $1 \rightarrow 2 \rightarrow 3$ (each the right child of the previous one) with $\alpha = 2/3$. Is a rebuild needed? If so, give the result.

4 becomes the right child of 3 at depth 3. $n = 4$, $\log_{1.5} 4 = 1.386 / 0.405 \approx 3.42$. Depth $3 \leq 3.42$, so **no rebuild** is triggered, even though root 1 is not α -weight-balanced (right size $3 > 2.67$). Insert 5 next: depth 4, $n = 5$, $\log_{1.5} 5 \approx 3.97 < 4 \rightarrow$ rebuild. Walk up: at 4, child size 1, size 2, $1 \leq 1.33$; at 3, child 2, size 3, $2 \leq 2$; at 2, child 3, size 4, $3 > 2.67 \rightarrow$ scapegoat 2. Rebuild $\{2, 3, 4, 5\}$: middle index $\lfloor (0+3)/2 \rfloor = 1 \rightarrow 3$ is the root, with 2 on the left and $4 \rightarrow 5$ on the right. Tree: $1(-, 3(2, 4(-, 5)))$.

Q. Build the trie for the keys car, cat, cart, dog. How many nodes (including the root)? Is "ca" a key?

Prefixes: c, ca, car, cat, cart, d, do, dog = 8 nodes + root = 9. car, cat, cart and dog are marked as key ends. "ca" exists as a path but is not marked, so it is not a key. car and cart share the path $c \rightarrow a \rightarrow r$; car is marked at "car", cart continues with t.

Q. Insert 5-bit keys $A = 10110$, $B = 10011$, $C = 00101$ (in this order) into an empty binary trie. Describe the tree.

A: bit 0 = 1, root's right is empty $\rightarrow$ leaf A on the right. B: bit 0 = 1 $\rightarrow$ reaches leaf A; compare 10110 and 10011: bit 0 same, bit 1 same (0), bit 2 differs (A 1, B 0). Create a node testing bit 1 (only child: left, bit 1 = 0), below it a node testing bit 2 with B on the left and A on the right. C: bit 0 = 0, root's left is empty $\rightarrow$ leaf C on the left. Left to right: C = 5, B = 19, A = 22, sorted $\checkmark$.

Q. An X-fast trie over 0 ... 15 stores 2, 9 and 13. List the level hash tables, then find $\text{succ}(10)$ and $\text{pred}(10)$.

$w = 4$. $2 = 0010$, $9 = 1001$, $13 = 1101$. $L_1 = \{0, 1\}$; $L_2 = \{00, 10, 11\}$; $L_3 = \{001, 100, 110\}$; $L_4 = \{0010, 1001, 1101\}$.

$10 = 1010$. Binary search on $\ell \in [0, 4]$: $\ell = 2$, "10" $\in L_2 \rightarrow$ longer; $\ell = 3$, "101" $\notin L_3 \rightarrow$ shorter; lowest ancestor = "10". The next bit is 1 and "10" has no right child, so its descendant pointer gives the largest leaf on its left, 9: $\text{pred}(10) = 9$. The next leaf is 13: $\text{succ}(10) = 13$.

Q. Why is successor in an X-fast trie $O(\log \log M)$ while insert is $O(\log M)$?

Successor needs only the lowest ancestor of k , found by binary search over the $w = \log M$ levels with $O(1)$ hash look-ups: $O(\log w) = O(\log \log M)$, then one descendant pointer and one linked-list step. Insert (and delete) must create (or delete) up to w internal nodes on the root-to-leaf path and fix their descendant pointers and hash tables: $O(w) = O(\log M)$.

Q. Compare X-fast and Y-fast tries (5 marks).

Both store integers from a bounded universe $0 \dots M - 1$ and give successor/predecessor in $O(\log \log M)$. X-fast: one trie over all keys, each level a hash table, find $O(1)$, insert/delete $O(\log M)$, space $O(n \log M)$. Y-fast: keys split into groups of $\Theta(\log M)$ stored in balanced BSTs, only one representative per group in an X-fast trie; find $O(\log \log M)$, insert/delete $O(\log \log M)$ amortized (split above $2 \log M$, merge below $(\log M)/4$), space $O(n)$. So Y-fast keeps the fast queries and cuts space to linear.

UNIT 12

File Structures

Data records ഒരു file-ൽ എങ്ങനെ സൂക്ഷിക്കുന്നു, എങ്ങനെ തിരികെ എടുക്കുന്നു എന്നതാണ് file organization. Sequential files (pile, sort), hashing ഉപയോഗിക്കുന്ന direct files, index ചേർത്ത indexed sequential files എന്നിവയും അവയുടെ ഗുണദോഷങ്ങളുമാണ് ഈ unit-ൽ.

File, data record and key

File organization and its types

Key drivers for file organization

Sequential files

Operations on sequential files (transaction log)

Types of sequential files: pile files and sort files

Sequential files: advantages, disadvantages and use cases

Direct file organization

Hash function and collision resolution

Direct files: advantages and disadvantages

Indexed sequential file organization

Static index and dynamic index

Indexed sequential files: advantages and disadvantages

Comparing the three file organizations

File, data record and key

File: a collection of related data records stored on a storage device.

Data record: the unit of a file that holds the data of a single entity, as a set of fields.

Key: a field (or set of fields) whose value uniquely identifies each record in the file.

File: ഒരു storage device-ൽ സൂക്ഷിച്ചിരിക്കുന്ന ബന്ധപ്പെട്ട data records-ന്റെ ശേഖരം.

Data record: ഒരു entity-യുടെ data മാത്രം fields ആയി സൂക്ഷിക്കുന്ന file-ന്റെ ഒരു unit.

Key: file-ലെ ഓരോ record-നെയും unique ആയി തിരിച്ചറിയാൻ സഹായിക്കുന്ന field (അല്ലെങ്കിൽ fields-ന്റെ കൂട്ടം).

Applications, operating systems and even databases (for example, transaction logs) keep their data in files. A file is made of records; a record is made of fields. One field, the key, is chosen so that no two records share its value. The five key file operations IGNOU lists are creation, update, deletion, merging and searching.

മലയാളത്തിൽ

ഒരു school register ഓർക്കുക. Register മുഴുവൻ ഒരു file; ഓരോ വരിയും ഒരു student-ന്റെ record; roll number ആണ് key, കാരണം രണ്ട് കുട്ടികൾക്ക് ഒരേ roll number ഉണ്ടാകില്ല. പേര് key ആക്കാൻ പറ്റില്ല, കാരണം ഒരേ ക്ലാസ്സിൽ രണ്ട് "Arun" ഉണ്ടാകാം.

File = collection of records | Record = collection of fields | Key = field that uniquely identifies a record

File operations: create, update (insert/modify records), delete, merge (combine several files into one), search (find a record)

EXAMPLE (IGNOU'S EMPLOYEE FILE)

Employee Number	Employee Name	Address	Department	Date of Join
45278	Employee 1	Kanpur	Sales	01-Jan-2020
167352	Employee 2	Mysore	Development	01-Mar-2020

Each row is one data record (one employee). Each column is a field. Employee Number is the key: it is unique for every employee, while name, address, department or date of join can repeat.

EXAMPLE

Define a data record for a university's student enrolment system and name its key.

Record: (Student Number, Student Name, Date of Enrolment, Address, Programme). Key: Student Number, because it is the only field guaranteed to be different for every student.

File organization and its types

File organization: the way data records are stored in a file and mapped to disk blocks, which decides how they are accessed and retrieved.

File organization: data records ഒരു file-ൽ എങ്ങനെ സൂക്ഷിക്കുന്നു, disk blocks-ലേക്ക് എങ്ങനെ map ചെയ്യുന്നു എന്നത്; ഇതാണ് records എങ്ങനെ access ചെയ്യാമെന്നും retrieve ചെയ്യാമെന്നും തീരുമാനിക്കുന്നത്.

A file can hold fixed-length records (every record the same size) or variable-length records (a flexible structure). The organization is chosen from the use case, the retrieval performance needed and how easy the data must be to manage. IGNOU names five common organizations.

മലയാളത്തിൽ

ഒരു വീട്ടിലെ സാധനങ്ങൾ എങ്ങനെ അടുക്കി വെക്കുന്നു എന്നതുപോലെയാണ് file organization. ഒരു പെട്ടിയിൽ എല്ലാം ക്രമത്തിൽ അടുക്കിയാൽ (sequential) ഒന്നിനു പുറകെ ഒന്നായി നോക്കണം; ഓരോന്നിനും നമ്പർ ഇട്ട അലമാര (direct) ആണെങ്കിൽ നേരെ എടുക്കാം; ക്രമത്തിൽ അടുക്കി മുകളിൽ ഒരു ലിസ്റ്റ് (index) ഒട്ടിച്ചാൽ രണ്ടും പറ്റും.

Organization	Idea	Access
Sequential file	Records stored in a pre-defined order, usually by key	Sequential only
Relative file	Record stored at a fixed position given by an integer key (record number)	Random (direct)
Direct file	Like a relative file, but the key can be non-integer; a hash function turns it into an address	Random (direct)
Indexed sequential file	Sequential file plus an index that gives random access	Both sequential and direct
Indexed file	Records reached through an index to speed up retrieval	Through the index

EXAMPLE

Each employee record is fixed length: number 6 bytes, name 20, address 20, department 12, date 11. A disk block holds 512 bytes. How many blocks does a file of 1000 employees need (records do not span blocks)?

Record size = $6 + 20 + 20 + 12 + 11 = 69$ bytes.

Records per block (blocking factor) = $\lfloor 512 / 69 \rfloor = 7$ ($7 \times 69 = 483 \leq 512$, $8 \times 69 = 552 > 512$).

Blocks = $\lceil 1000 / 7 \rceil = 143$ (142 blocks hold only 994 records).

With fixed-length records, record number r (starting at 0) sits in block $\lceil r / 7 \rceil$, which is exactly what makes relative (record-number) access possible.

Key drivers for file organization

Drivers of file organization: the goals a file organization is chosen for: fast retrieval, efficient insert/update/delete, data consistency without duplicates, and optimal storage use.

Drivers of file organization: ഒരു file organization തിരഞ്ഞെടുക്കുന്നതിന്റെ ലക്ഷ്യങ്ങൾ: വേഗത്തിലുള്ള retrieval, കാര്യക്ഷമമായ insert/update/delete, duplicates ഇല്ലാത്ത data consistency, storage-ന്റെ മികച്ച ഉപയോഗം.

IGNOU lists four drivers: (1) design records so they can be retrieved as quickly as possible, (2) improve performance of insert, update and delete, (3) keep data consistent and avoid duplicates during file operations, (4) use storage optimally. No single organization wins on all four, which is why several exist.

മലയാളത്തിൽ

ഒരു കടയിലെ സ്റ്റോക്ക് ഓർക്കുക: സാധനം വേഗം കണ്ടെത്തണം, പുതിയ സാധനം എളുപ്പം വെക്കാൻ പറ്റണം, ഒരേ സാധനം രണ്ടിടത്ത് എണ്ണിപ്പോകരുത്, സ്ഥലം പാഴാകരുത്. ഈ നാല് ആവശ്യങ്ങളും ഒരുമിച്ച് പൂർണ്ണമായി നേടാൻ പറ്റില്ല, അതുകൊണ്ടാണ് പല file organizations ഉള്ളത്.

1. Fast retrieval
2. Fast insert / update / delete
3. Data consistency, no duplicates
4. Optimal storage

Exam tip: For a "why do we need file organization?" question, list these four drivers and add one line on the trade-off: sequential files win on storage, direct files win on retrieval speed.

Sequential files

Sequential file: a file in which records are stored one after another in a specific order (usually by key) and are accessed in that same order.

Sequential file: records ഒരു പ്രത്യേക ക്രമത്തിൽ (സാധാരണ key അനുസരിച്ച്) ഒന്നിനു പുറകെ ഒന്നായി സൂക്ഷിക്കുകയും അതേ ക്രമത്തിൽ access ചെയ്യുകയും ചെയ്യുന്ന file.

Records are read in the order they were written to the device: reading starts at the beginning of the file and writing happens at the end. To reach record number k you must pass over the $k - 1$ records before it. Legacy storage such as magnetic tape uses sequential files.

മലയാളത്തിൽ

പഴയ cassette tape-ൽ അഞ്ചാമത്തെ പാട്ട് കേൾക്കണമെങ്കിൽ ആദ്യത്തെ നാല് പാട്ടുകളും forward ചെയ്ത് കടക്കണം. Sequential file-ഉം അങ്ങനെയാണ്: ഒരു record-ലേക്ക് നേരെ ചാടാൻ പറ്റില്ല, തുടക്കം മുതൽ വായിച്ചു പോകണം.

Successful search of n records: average $(n + 1)/2$ comparisons, worst n

Unsuccessful search: n comparisons (unsorted); in a sorted file you can stop at the first key larger than the target

EXAMPLE

A sequential file on tape holds 1000 records. On average how many records must be read to find one that exists?

Each position 1 to 1000 is equally likely, so the average is $(1 + 2 + \dots + 1000)/1000 = (1000 + 1)/2 = 500.5$ reads. Worst case: 1000 reads.

EXAMPLE

Sorted file keys: 10, 20, 30, 40, 50. Search for 35.

Compare 10, 20, 30 (all smaller), then $40 > 35$: stop. 35 is not present, found after 4 comparisons instead of 5, because the file is sorted.

Operations on sequential files (transaction log)

Transaction log: a separate file that collects pending inserts, deletes and updates, sorted by key, to be merged into the main (master) file later in batch mode.

Transaction log: pending ആയ inserts, deletes, updates എന്നിവ key ക്രമത്തിൽ ശേഖരിച്ച്, പിന്നീട് batch mode-ൽ main (master) file-ലേക്ക് merge ചെയ്യാനുള്ള വേറൊരു file.

Changing a sorted sequential file directly is expensive, because one insert can shift every later record. So changes are collected in a transaction log, sorted by key, and merged into the main file in one pass.

- **Insert:** find the correct key position; for many inserts, write them to the log, sort the log, then merge.
- **Delete:** put a delete marker for the key in the log; during the merge the marked record is dropped and its space reclaimed.
- **Update:** a delete marker for the old record plus an insert of the new record for the same key, both in the log.
- **Retrieval:** a stale read takes the record straight from the main file; a consistent read must also apply any pending changes for that key from the log.

മലയാളത്തിൽ

ഒരു ബാക്ക് passbook ഓർക്കുക. ഓരോ ചെറിയ മാറ്റത്തിനും ledger മുഴുവൻ വീണ്ടും എഴുതില്ല; ദിവസത്തെ എല്ലാ transactions-ഉം ഒരു പ്രത്യേക പുസ്തകത്തിൽ (transaction log) കുറിച്ച്, വൈകുന്നേരം ഒറ്റയടിക്ക് ledger-ലേക്ക് ചേർക്കും. Delete ചെയ്യാൻ record-ന് ഒരു "delete marker" ഇടും, update എന്നാൽ പഴയത് delete ചെയ്ത് പുതിയത് insert ചെയ്യുന്നതാണ്.

Update = delete marker (old record) + insert (new record)

Batch merge: one pass over the sorted master file and the sorted log, like merging two sorted lists

EXAMPLE

Master file keys: 10, 20, 30, 40, 50. Pending changes in arrival order: insert 55, delete 40, update 20 (new data 20'), insert 25. Show the sorted log and the new master file.

Step 1: write each change to the log. The update becomes two entries: "20 delete" and "20' insert".

Step 2: sort the log by key: 20 delete, 20' insert, 25 insert, 40 delete, 55 insert.

Step 3: merge in one pass, walking both files from the start:

Master record	Log entry	Action	Output so far
10	20 delete	$10 < 20$, copy 10	10
20	20 delete, 20' insert	drop old 20, write 20'	10, 20'
30	25 insert	$25 < 30$, write 25	10, 20', 25
30	40 delete	$30 < 40$, copy 30	10, 20', 25, 30
40	40 delete	drop 40	10, 20', 25, 30
50	55 insert	$50 < 55$, copy 50	10, 20', 25, 30, 50
(end)	55 insert	write 55	10, 20', 25, 30, 50, 55

New master file: 10, 20', 25, 30, 50, 55. Every record was moved once, instead of shifting records for each change separately.

EXAMPLE

Before the merge above, a user reads key 40. What does a stale read return, and what does a consistent read return?

Stale read: the main file still has 40, so it returns the old record. Consistent read: the log has "40 delete", so the correct answer is "record not found".

Types of sequential files: pile files and sort files

Pile file: a sequential file in which records are stored in the order of their arrival; new records are added at the end.

Sort file (sorted file): a sequential file in which records are kept sorted on the key; a new record is inserted at its key position.

Pile file: records വന്ന ക്രമത്തിൽ തന്നെ സൂക്ഷിക്കുന്ന sequential file; പുതിയ record അവസാനം ചേർക്കുന്നു.

Sort file (sorted file): records key അനുസരിച്ച് sorted ആയി സൂക്ഷിക്കുന്ന sequential file; പുതിയ record അതിന്റെ key-യുടെ സ്ഥാനത്ത് insert ചെയ്യുന്നു.

In a pile file, insert is cheap (append at the end) but search always scans from the start, and delete means finding the record and putting a deletion marker on it. In a sort file, search can stop early and records come out in key order, but each insert must find its place and shift the later records.

മലയാളത്തിൽ

Pile file ഒരു കൂട്ടം പത്രങ്ങൾ വന്ന ക്രമത്തിൽ മേശപ്പുറത്ത് അടുക്കുന്നതുപോലെ:

വെക്കാൻ എളുപ്പം, പക്ഷേ ഒരു പ്രത്യേക തീയതിയുടേത് തിരയാൻ മുഴുവൻ നോക്കണം.

Sort file തീയതി ക്രമത്തിൽ അടുക്കിയ പത്രക്കെട്ട് പോലെ: തിരയാൻ എളുപ്പം, പക്ഷേ

വൈകി വന്ന ഒരു പത്രം നടുവിൽ തിരക്കി വെക്കാൻ ബാക്കിയെല്ലാം നീക്കണം.

EXAMPLE (IGNOU FIGURES 2 TO 5)

Records arrive in the order DR1, DR3, DR5, DR2. Then DR4 is inserted. Show the file for both types.

	Pile file	Sort file
After DR1, DR3, DR5, DR2	DR1, DR3, DR5, DR2	DR1, DR2, DR3, DR5
Insert DR4	Append at end: DR1, DR3, DR5, DR2, DR4	Find position between DR3 and DR5, shift DR5 one place: DR1, DR2, DR3, DR4, DR5
Records moved by the insert	0	1 (DR5)

EXAMPLE

Using the files after inserting DR4, how many comparisons does an unsuccessful search for DR6 take? And for "DR2.5" (a key between DR2 and DR3)?

DR6 in the pile file: all 5 records are compared (5). In the sort file: all 5 are smaller than DR6, so also 5.

DR2.5 in the pile file: still 5, because the pile has no order. In the sort file: compare DR1, DR2, then DR3 > DR2.5, stop after 3 comparisons.

Operation	Pile file	Sort file
Insert	Fast, append at end	Slow, find place and shift
Search	Scan from start till found	Scan, but can stop early
Delete	Search, then set deletion marker	Search, then set deletion marker
Read in key order	Needs sorting first	Direct, already ordered

Sequential files: advantages, disadvantages and use cases

Sequential files in short: simple, storage-efficient and cheap, but slow, because a record can only be reached by reading all records before it.

Sequential files ചുരുക്കത്തിൽ: ലളിതവും storage കാര്യക്ഷമവും ചെലവു കുറഞ്ഞതും, പക്ഷേ മുമ്പുള്ള എല്ലാ records-ഉം വായിച്ചാൽ മാത്രമേ ഒരു record-ൽ എത്തൂ എന്നതിനാൽ വേഗത കുറവ്.

Sequential files suit jobs that touch most records anyway (batch processing such as monthly payroll or billing) and long-term archival, where large volumes are stored cheaply and slow retrieval is acceptable.

മലയാളത്തിൽ

മാസാവസാനം എല്ലാ ജീവനക്കാർക്കും salary കണക്കാക്കുമ്പോൾ എല്ലാ records-ഉം ക്രമത്തിൽ വായിക്കണം, അവിടെ sequential file തന്നെ മികച്ചത്. പക്ഷേ ഒരു customer ബാങ്കിൽ വന്ന് balance ചോദിക്കുമ്പോൾ ലക്ഷക്കണക്കിന് records തുടക്കം മുതൽ വായിക്കുന്നത് വളരെ മന്ദമാണ്.

Advantages	Disadvantages
1. Simple and easy to implement	1. Update is costly and time consuming
2. Best use of storage space	2. Access is slow (sequential access)
3. Suitable for data archival	3. A record cannot be accessed directly by key
4. Low-cost storage option	4. All records need a uniform structure
5. Can use inexpensive devices like tape	5. Insert into a sorted file is slow
	6. Overall processing is slow

Use cases: long-term archival, batch processing (payroll, billing, end-of-day reports), large data that tolerates slow retrieval

EXAMPLE

Choose sequential or not: (a) a telephone company printing all monthly bills; (b) an ATM checking one account's balance.

(a) Sequential: every record is processed once, in order, so reading straight through is the fastest possible.

(b) Not sequential: only one record is needed and must be found at once, so a direct or indexed organization is better.

Direct file organization

Direct (random) file organization: records are stored at addresses computed from their keys by a hash function, so any record can be accessed directly without reading the others.

Direct (random) file organization: ഓരോ record-ഉം അതിന്റെ key-ൽ നിന്ന് hash function കണക്കാക്കുന്ന address-ൽ സൂക്ഷിക്കുന്നു; അതിനാൽ മറ്റു records വായിക്കാതെ ഏതു record-ഉം നേരെ access ചെയ്യാം.

To access a record directly we must map its key to the address where it is stored. The records are spread over a direct access storage device (DASD) such as a hard disk, magnetic disk or CD; tape cannot be used. Because the address is computed, not searched for, direct access is much faster than sequential access.

മലയാളത്തിൽ

ഒരു hostel-ൽ ഓരോ കുട്ടിയുടെയും room number അവരുടെ admission number-ൽ നിന്ന് ഒരു നിയമം വെച്ച് കണക്കാക്കുന്നു എന്ന് വിചാരിക്കുക. ഒരാളെ കാണാൻ എല്ലാ മുറിയിലും മുട്ടേണ്ട; കണക്ക് ചെയ്ത് നേരെ ആ മുറിയിലേക്ക് പോകാം. ഇതാണ് direct file: key $\rightarrow$ hash function $\rightarrow$ address.

key $\rightarrow$ hash function $\rightarrow$ disk address (bucket) $\rightarrow$ record
Storage: DASD (hard disk, magnetic disk, CD), never tape

EXAMPLE

A direct file has 7 buckets (0 to 6) and uses $h(\text{key}) = \text{key} \bmod 7$. Where are employees 45278 and 167352 stored?

$45278 = 7 \times 6468 + 2$, so $h(45278) = 2 \rightarrow$ bucket 2.

$167352 = 7 \times 23907 + 3$, so $h(167352) = 3 \rightarrow$ bucket 3.

To read employee 167352 later, compute $167352 \bmod 7 = 3$ and read bucket 3 only: one disk access.

Hash function and collision resolution

Hash function: a function that converts a record's key into the address (hash value) where the record is stored; the common one is $h(k) = k \bmod m$.

Collision: the situation where two different keys produce the same hash value.

Collision resolution: separate chaining links colliding records in a list at the same slot; open addressing probes for the next free slot.

Hash function: record-ന്റെ key-യെ അത്സൂക്ഷിക്കുന്ന address (hash value) ആക്കി മാറ്റുന്ന function; സാധാരണ ഉപയോഗിക്കുന്നത് $h(k) = k \bmod m$.

Collision: രണ്ട് വ്യത്യസ്ത keys-ന് ഒരേ hash value കിട്ടുന്ന അവസ്ഥ.

Collision resolution: separate chaining ഒരേ slot-ൽ വരുന്ന records-നെ ഒരു list ആയി ബന്ധിപ്പിക്കുന്നു; open addressing അടുത്ത ഒഴിഞ്ഞ slot probe ചെയ്ത് കണ്ടെത്തുന്നു.

The modulo (division) hash takes the remainder of the key divided by the table size m , so every key lands in 0 to $m - 1$. With many keys and few slots, collisions are unavoidable. Linear probing, the simplest open-addressing rule, tries $h, h + 1, h + 2, \dots \pmod{m}$ until it finds an empty slot.

മലയാളത്തിൽ

Parking-ൽ നിങ്ങൾക്ക് കിട്ടിയ slot-ൽ ആരോ വണ്ടി ഇട്ടിട്ടുണ്ടെങ്കിൽ എന്ത് ചെയ്യും? അടുത്ത ഒഴിഞ്ഞ slot നോക്കി ഇടും, അതാണ് open addressing (linear probing). അല്ലെങ്കിൽ അതേ slot-ൽ ഒന്നിനു പുറകിൽ ഒന്നായി നിരത്തി ഇടും, അതാണ് separate chaining.

Division hash: $h(k) = k \bmod m$

Linear probing: try $(h(k) + i) \bmod m$, for $i = 0, 1, 2, \dots$

Separate chaining: slot $h(k)$ holds a linked list of all keys with that hash value

EXAMPLE (OPEN ADDRESSING, LINEAR PROBING)

Insert keys 50, 700, 76, 85, 92, 73, 101 into a table of 7 slots with $h(k) = k \bmod 7$.

Key	$k \bmod 7$	Probes	Stored in slot
50	1	1	1
700	0	0	0
76	6	6	6
85	1	1 (full), 2	2
92	1	1, 2 (full), 3	3
73	3	3 (full), 4	4
101	3	3, 4 (full), 5	5

Final table: [0] 700, [1] 50, [2] 85, [3] 92, [4] 73, [5] 101, [6] 76.

EXAMPLE (SEPARATE CHAINING)

Insert the same keys using separate chaining.

```
slot 0: 700
slot 1: 50 → 85 → 92
slot 2: (empty)
slot 3: 73 → 101
slot 4: (empty)
slot 5: (empty)
slot 6: 76
```

Searching for 92: compute $92 \bmod 7 = 1$, then walk the chain at slot 1: 50, 85, 92, found in 3 comparisons.

Exam tip: Choose m as a prime not close to a power of 2; keys then spread more evenly over the buckets. Hashing is covered in detail in Unit 10.

Direct files: advantages and disadvantages

Direct files in short: very fast key-based access, insert, update and delete, at the price of costlier DASD storage and wasted space in unused buckets.

Direct files ചുരുക്കത്തിൽ: key ഉപയോഗിച്ചുള്ള access, insert, update, delete എന്നിവ വളരെ വേഗം; പക്ഷേ ചെലവേറിയ DASD storage വേണം, ഉപയോഗിക്കാത്ത buckets-ൽ space പാഴാകും.

Because the address is computed from the key, direct files suit online systems where one record must be found immediately: databases, reservations, ATMs. They are poor for reading all records in key order, since hashing scatters the keys.

മലയാളത്തിൽ

Railway reservation-ൽ ഒരു PNR number അടിച്ചാൽ ഉടൻ ticket വിവരം വരണം, അവിടെ direct file ആണ് ഉചിതം. പക്ഷേ എല്ലാ PNR-ഉം ക്രമത്തിൽ print ചെയ്യണമെങ്കിൽ hashing കാരണം records ചിതറിക്കിടക്കുന്നതിനാൽ ആദ്യം sort ചെയ്യേണ്ടി വരും.

Advantages	Disadvantages
1. Fast retrieval; suits transaction systems such as relational databases	1. Needs costlier storage technology (DASD)
2. Fast insert, update and delete	2. Storage use is sub-optimal (empty buckets, overflow)
3. Efficient storage and retrieval of large data	3. Collisions must be handled; inserts, deletes and updates must keep the key-to-address mapping correct
4. Fast key-based search	4. Records are not in key order, so ordered (sequential) processing is slow
5. Suits real-time transactions	
6. Concurrent processing is possible	

Exam tip: IGNOU's third disadvantage says "insert, delete and update needs update of the index table". A pure direct (hashed) file has no index table; the address comes from the hash function. The accurate point is that every change must keep the key-to-address mapping (hash buckets and collision chains) correct. Write it that way; if you quote IGNOU's wording, add this clarification. Disadvantage 4 is a standard textbook point added here.

Indexed sequential file organization

Indexed sequential file (ISAM): a file whose records are stored sequentially by key and which also has an index on the key, so records can be accessed both directly (through the index) and sequentially.

Indexed sequential file (ISAM): records key ക്രമത്തിൽ sequential ആയി സൂക്ഷിക്കുകയും key-ൽ ഒരു index കൂടി ഉണ്ടാവുകയും ചെയ്യുന്ന file; അതിനാൽ records index വഴി direct ആയും sequential ആയും access ചെയ്യാം.

The index file and the data file are kept separately. The index is hierarchical (tree-like); the data file stores records sequentially in blocks. In IGNOU's two-level picture, level 1 (the index) is accessed directly and each index entry points to the first record of a block; level 2 (the data) is then read sequentially from that record. Because of the direct index access, these files need a random access device such as a magnetic disk.

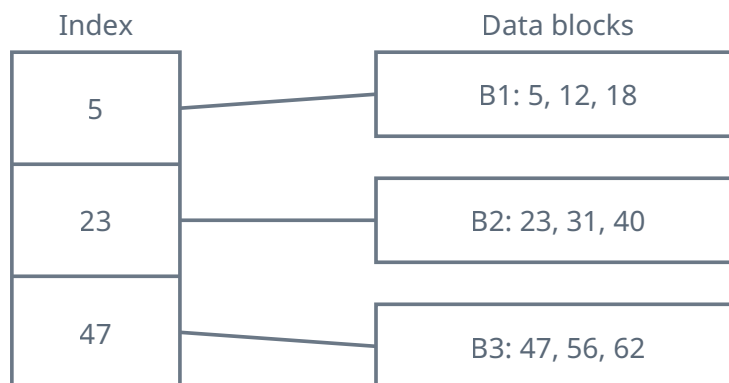
മലയാളത്തിൽ

ഒരു നിഘണ്ടു ഓർക്കുക. ഓരോ പേജിന്റെയും മുകളിൽ ആ പേജിലെ ആദ്യത്തെ വാക്ക് കൊടുത്തിട്ടുണ്ട് (index). "Kerala" എന്ന വാക്ക് തിരയാൻ മുകളിലെ വാക്കുകൾ നോക്കി ശരിയായ പേജിലേക്ക് നേരെ ചാടുന്നു (direct), എന്നിട്ട് ആ പേജിൽ മുകളിൽ നിന്ന് താഴേക്ക് വായിക്കുന്നു (sequential). ഇതാണ് indexed sequential file. IGNOU-ന്റെ ഉദാഹരണം: country record index വഴി നേരെ കണ്ടെത്തി, അത് ആ രാജ്യത്തെ states-ന്റെ sequential file-ലേക്ക് point ചെയ്യുന്നു.

Level 1 (index): accessed directly; each entry = (first key of block, pointer to block)

Level 2 (data): records in key order, read sequentially

Search k: find the last index entry with key $\leq k$, jump to that block, scan it



Level 1 index entries point to the first record of each sequential data block.

EXAMPLE

Using the file in the figure, search for key 31, then key 20. Compare with a plain sequential search.

Key 31: index entries $5 \leq 31$, $23 \leq 31$, $47 > 31$, so the last entry ≤ 31 is 23 $\rightarrow$ block B2. Scan B2: 23, 31, found. Records compared: 2. A plain sequential search would compare 5, 12, 18, 23, 31: 5 records.

Key 20: last index entry ≤ 20 is 5 $\rightarrow$ block B1. Scan 5, 12, 18; the block ends with $18 < 20$ and the next block starts at 23, so 20 is not in the file. Only B1 was read.

EXAMPLE

A file of 1,000,000 records is stored in 1000 blocks of 1000 records each, with a sorted index of 1000 entries. Roughly how many comparisons does a successful search need?

Binary search of the index: about $\lceil \log_2 1000 \rceil = 10$ comparisons. Then an average of $(1000 + 1)/2 \approx 500$ comparisons inside the block. Total ≈ 510 , against $\approx 500,000$ for a sequential search of the whole file.

Exam tip: Remember the pairing IGNOU asks as a fill-in: the first level is accessed directly, the second level sequentially.

Static index and dynamic index

Static index: an index whose structure does not change when records in the data file are inserted, deleted or updated (new records go to overflow areas).

Dynamic index: an index whose structure changes (grows, splits, shrinks) as records in the data file are inserted, deleted or updated.

Static index: data file-ലെ records insert, delete, update ചെയ്താലും structure മാറാത്ത index (പുതിയ records overflow area-യിലേക്ക് പോകും).

Dynamic index: data file-ലെ records insert, delete, update ചെയ്യുമ്പോൾ structure മാറുന്ന (വളരുന്ന, split ആകുന്ന, ചുരുങ്ങുന്ന) index.

When a record is inserted into an indexed sequential file, the data file is updated in key order and the index updated if needed. With a static index (classic ISAM) the index is built once; later inserts that do not fit their block are put in overflow chains, so performance slowly degrades until the file is reorganized. With a dynamic index (for example a B+ tree) the index reshapes itself on every change and stays balanced.

മലയാളത്തിൽ

ഒരു പുസ്തകത്തിന്റെ അച്ചടിച്ച ഉള്ളടക്ക പട്ടിക static index പോലെയാണ്: പുതിയ പേജുകൾ ചേർത്താലും പട്ടിക മാറില്ല, പുതിയവ അവസാനം ഒരു അനുബന്ധമായി (overflow) ചേർക്കും. Phone-ലെ contact list dynamic index പോലെയാണ്: പുതിയ പേര് ചേർക്കുമ്പോൾ തന്നെ ലിസ്റ്റ് സ്വയം പുനഃക്രമീകരിക്കും.

	Static index	Dynamic index
Index structure on updates	Unchanged	Changes with the data
New records that do not fit	Overflow area / chains	Blocks split, index adjusted
Performance over time	Degrades until reorganization	Stays stable
Typical example	Classic ISAM	B+ tree

EXAMPLE

In the figure of the previous topic each block holds 3 records and all are full. Insert key 15 with (a) a static index, (b) a dynamic index.

(a) Static: 15 belongs in B1 ($5 \leq 15 < 23$) but B1 is full. The index stays 5, 23, 47; 15 goes to an overflow record chained from B1. Searching 15 now means reading B1 and then the overflow chain.

(b) Dynamic: B1 with 15 would be 5, 12, 15, 18, so it splits into B1 = 5, 12 and B1' = 15, 18. A new index entry $15 \rightarrow B1'$ is added: the index becomes 5, 15, 23, 47.

Indexed sequential files: advantages and disadvantages

Indexed sequential files in short: fast direct access plus ordered sequential access, at the cost of extra storage for the index and the need for random access hardware.

Indexed sequential files ചുരുക്കത്തിൽ: വേഗത്തിലുള്ള direct access-ഉം ക്രമത്തിലുള്ള sequential access-ഉം ലഭിക്കും; പക്ഷേ index-ന് അധിക storage-ഉം random access hardware-ഉം വേണം.

The index lets you jump close to a record, so single-record lookups are fast, and the data is still in key order for batch jobs. The price is the index itself, overflow or wasted space in blocks, and disk hardware.

മലയാളത്തിൽ

ഒരു library-യിൽ പുസ്തകങ്ങൾ ക്രമത്തിൽ അടുക്കി, ഒരു catalogue-ഉം വെച്ചാൽ ഒരു പുസ്തകം പെട്ടെന്ന് കണ്ടെത്താം, എല്ലാം ക്രമത്തിൽ എണ്ണാനും പറ്റും. പക്ഷേ catalogue-ന് വേറെ സ്ഥലവും അത് പുതുക്കി വെക്കാനുള്ള പണിയും വേണം.

Advantages	Disadvantages
1. Records are reached directly through the index, so access is fast	1. Storage use is sub-optimal (overflow areas, partly empty blocks)
2. Supports both direct and sequential processing of the same file	2. Costly to implement; needs random access hardware
3. Record insert is simpler than in a sorted sequential file (only one block and possibly the index change)	3. Extra storage is needed for the index file
	4. With a static index, many inserts create overflow chains and slow the file until it is reorganized

Exam tip: IGNOU lists "record insert is very fast" as an advantage. That is only true compared with a sorted sequential file, where an insert can shift every later record. Compared with a direct (hashed) file, ISAM inserts are slower, since the record must go into its key position in a block, possibly into an overflow area, and the index may need updating. Write "insert is faster than in a sorted sequential file" to be safe. Advantage 2 and disadvantage 4 are standard textbook points added here.

Comparing the three file organizations

Choosing an organization: sequential for batch processing and archival, direct for fast single-record access by key, indexed sequential when both direct and ordered access are needed.

Organization തിരഞ്ഞെടുക്കൽ: batch processing-നും archival-നും sequential, key വഴി ഒരു record വേഗം എടുക്കാൻ direct, direct access-ഉം ക്രമത്തിലുള്ള access-ഉം രണ്ടും വേണമെങ്കിൽ indexed sequential.

This table pulls the unit together and answers the common "compare" and "which organization would you use" questions.

മലയാളത്തിൽ

ഏത് organization വേണം എന്നത് ചോദ്യം നോക്കി തീരുമാനിക്കുക: "എല്ലാ records-ഉം ഒരുമിച്ച്" എന്നാൽ sequential, "ഒരു record ഉടനെ" എന്നാൽ direct, "രണ്ടും" എന്നാൽ indexed sequential.

Feature	Sequential	Direct	Indexed sequential
Access	Sequential only	Direct (random) by key	Direct and sequential
How a record is located	Scan from start	Hash function on key	Index, then scan block
Storage device	Tape or disk	DASD (disk)	DASD (disk)
Storage use	Best	Sub-optimal	Sub-optimal (index + overflow)
Single-record search	Slow, $O(n)$	Fastest, about 1 access	Fast
Processing in key order	Fast	Slow (keys scattered)	Fast
Typical use	Payroll, billing, archival	Reservations, ATM, databases	Hierarchical data, systems needing both lookup and reports

EXAMPLE

A university keeps student records. It must (a) show one student's result online the moment a roll number is typed, and (b) print a merit list of all students in roll-number order each semester. Which organization?

Indexed sequential. The index gives fast lookup for (a), and the data file is already in roll-number order for (b). A direct file would handle (a) but make (b) slow; a sequential file would handle (b) but make (a) slow.

Quick recap

- A file is a collection of data records; a record holds the data of one entity; a key uniquely identifies each record.
- File operations: create, update, delete, merge, search.
- File organization is how records are stored and mapped to disk blocks; records may be fixed or variable length.
- Types: sequential, relative, direct, indexed sequential, indexed.
- Drivers: fast retrieval, fast insert/update/delete, consistency without duplicates, optimal storage.
- Sequential file: records in key order, read from the start, written at the end; used on tape.
- Changes to sequential files are collected in a sorted transaction log and merged in batch; update = delete marker + insert.
- Pile file: records in arrival order, insert at end. Sort file: records in key order, insert at key position.
- Sequential files: simple, cheap, storage-efficient, but slow and without direct access; best for archival and batch jobs.

- Direct file: a hash function maps the key to an address on a DASD; fastest single-record access.
- Common hash: $h(k) = k \bmod m$. Collisions are resolved by separate chaining or open addressing (probing).
- Indexed sequential file: separate index (accessed directly) and sequential data file (read sequentially); needs a random access device.
- Static index keeps its structure (overflow areas); dynamic index changes with the data (for example B+ tree).
- ISAM costs extra storage for the index and random access hardware.

Check yourself

Q. Fill in the blanks: (a) A _____ in a file encapsulates the data of a single entity. (b) Each data record is uniquely identified by a _____.

(a) Data record. (b) Key.

Q. Fill in the blanks: (a) In _____ files, records are inserted in the order of their arrival. (b) The two types of sequential files are _____ and _____.

(a) Pile files. (b) Pile files and sort (sorted) files.

Q. To optimize performance, all updates to a sequential file are collected in a _____ and later merged into the main file. Explain how an update is recorded there.

Transaction log. An update is written as a delete marker for the old record plus an insert of the new record with the same key. The sorted log is merged into the main file in batch mode.

Q. Define a data record for a typical university's student enrolment system.

(Student Number, Student Name, Date of Enrolment, Address, Programme), with Student Number as the key because it is unique for every student.

Q. Fill in the blanks: (a) In direct file organization we access the file directly by its _____. (b) The function that converts the key to an address is called a _____. (c) Records of a direct file are stored on a _____ storage device.

(a) Key. (b) Hash function. (c) Direct access (DASD), such as a hard disk.

Q. In indexed sequential access, the first level is accessed ____ and the second level _____. Which file organization is used for managing hierarchical data?

Directly; sequentially. Indexed sequential file organization is used for hierarchical data (for example, a country record found through the index points to the sequential file of its states).

Q. Records arrive as DR4, DR1, DR3, then DR2 is inserted. Show the file as a pile file and as a sort file.

Pile file: DR4, DR1, DR3, DR2 (arrival order, DR2 appended at the end).

Sort file: after DR4, DR1, DR3 the file is DR1, DR3, DR4. DR2 goes between DR1 and DR3, so DR3 and DR4 shift: DR1, DR2, DR3, DR4.

Q. Master file: 5, 15, 25, 35. Log (unsorted): insert 30, delete 5, insert 10. Give the sorted log and the new master file.

Sorted log: 5 delete, 10 insert, 30 insert.

Merge: drop 5, write 10, copy 15, copy 25, write 30 ($30 < 35$), copy 35. New master: 10, 15, 25, 30, 35.

Q. Insert 18, 41, 22, 44, 59, 32 into a direct file of 11 buckets with $h(k) = k \bmod 11$ and linear probing. Show the final table.

$18 \bmod 11 = 7 \rightarrow$ slot 7. $41 \bmod 11 = 8 \rightarrow$ slot 8. $22 \bmod 11 = 0 \rightarrow$ slot 0. $44 \bmod 11 = 0$, slot 0 full $\rightarrow$ slot 1. $59 \bmod 11 = 4 \rightarrow$ slot 4. $32 \bmod 11 = 10 \rightarrow$ slot 10.

Final: [0] 22, [1] 44, [4] 59, [7] 18, [8] 41, [10] 32; slots 2, 3, 5, 6, 9 empty. One collision (44 with 22).

Q. Compare sequential, direct and indexed sequential file organizations, giving one use case for each.

Sequential: records in key order, read from the start; simple, cheap, best storage use, but slow single-record access. Use: monthly payroll or archival on tape.

Direct: a hash function maps the key to a disk address; fastest single-record access, fast insert/update/delete, but costlier DASD storage, wasted space and poor ordered processing. Use: railway reservation lookup by PNR.

Indexed sequential: sorted data file plus an index; supports both direct and sequential access, but needs extra index storage and random access hardware. Use: a student result system needing both instant lookup and sorted merit lists.

Based on the IGNOU unit texts. Check definitions against the PDFs before the exam.